

DATA STRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	Katakam Satya Mohit
ROLL NO.	AV.SC.U4CSE25206
YEAR/SEM/SECTION	2nd YEAR/3rd SEM/CSE-C
Credits	05

Submitted to	Dr. GAYATHRI PARASA
Designation	Asst. Professor (Sr.Gr)
Department	School of Computing

S.NO	TITLE NAME	PAGE NO-
1	a) A rocket launch system displays a count down before launch. Instead of using a loop, implement the count down using recursion. Write a recursive python program to display countdown from n to 1 followed by launch. b) Banks calculate the growth of investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n where P = Principal growth factor, n = number of years c) A HR department stores employee details in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. d) In a logistic department, the no of possible ways to arrange n diff parcels are calculated using the factorial formula(n!). Write a recursive formula to find the factorial of a given number e) Print first n terms in a fibanocci number series	5-8
2	Write the implementation for the linear search algorithm Write the implementation for the binary search algorithm	9-10
3	a) Write a python program to implement bubble sort b) Write a python program to implement insertion sort c) Write a python program to implement selection sort	11-13
4	a) Write a python program to implement merge sort b) Write a python program to implement quick sort	14-15

5	a) Write a Python program to implement a Singly Linked List (SLL) with the following operations: 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index 5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Sort the data of SLL 10) Search data in SLL 11) Display or Traverse 12)Exit	16-19
6	Write a Python program to implement a Double Linked List (DLL) with the following operations: 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index 5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Display or Traverse 10)Exit	20-25
7	Write a python program to implement stack operations such as push, pop, peek, display using an array. Write a python program to implement stack operations such as push, pop, peek, display using a linked list.	26-28
8	Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures. Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.	29-34
9	Write a python program to implement a Binary Search Tree (BST) and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals – in-order, pre-order and post-order	35-38

	4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	
10	Write a menu driven python program to implement an AVL Tree and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	39-41
11	Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations: 1. Insertion of an element into the Heap. 2. Deletion of the root element from the Heap. 3. Display the array representation of the Heap. 4. Exit the program.	42-43
12	Write a program to implement the following three collision resolution techniques in a Hash Table: 1. Separate Chaining 2. Linear Probing 3. Quadratic Probing	44-47

WEEK 1

Aim:

Recursion:

a)A rocket launch system displays a count down before launch. Instead of using a loop, implement the count down using recursion. Write a recursive python program to display countdown from n to 1 followed by launch.

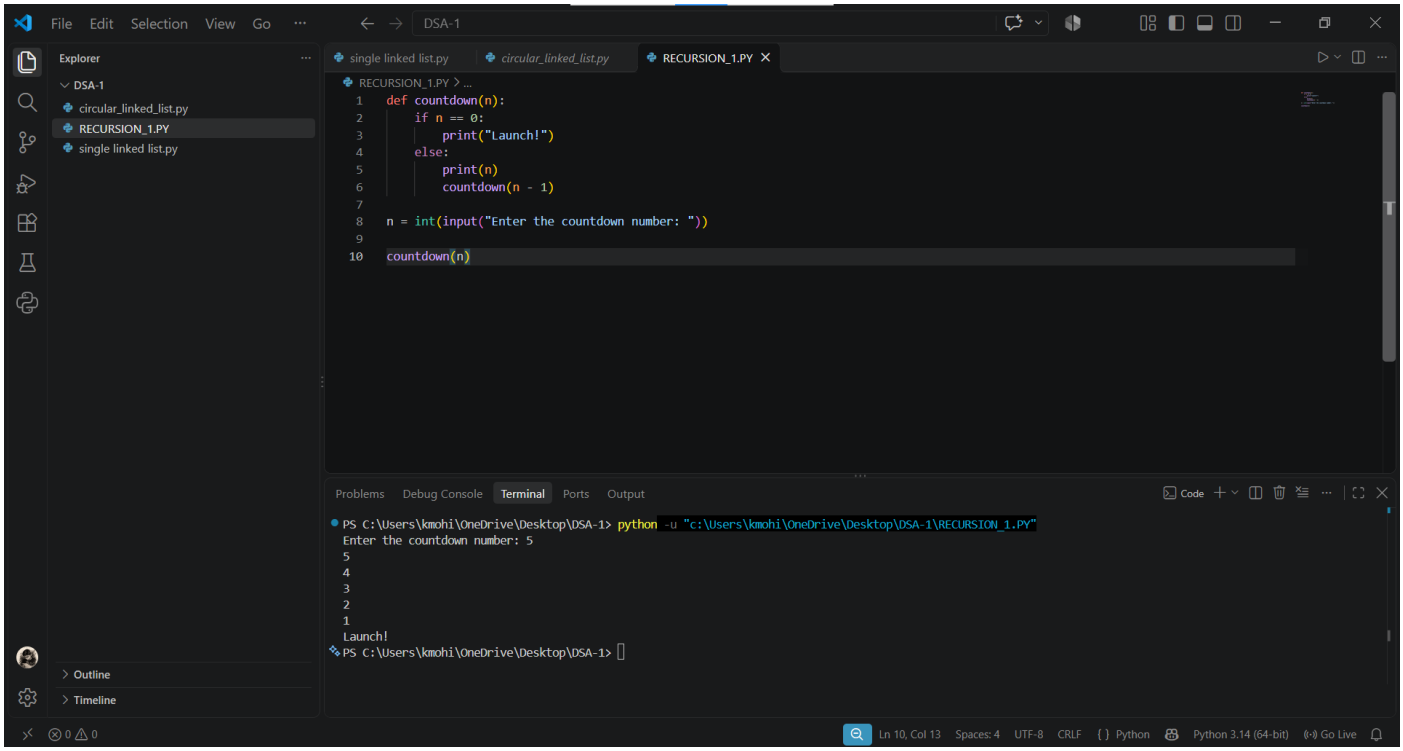
Program:

```
def countdown(n):
    if n == 0:
        print("Launch!")
    else:
        print(n)
        countdown(n - 1)

n = int(input("Enter the countdown number: "))

countdown(n)
```

Output:



b) Banks calculate the growth of investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n where P = Principal growth factor, n = number of years

Program:

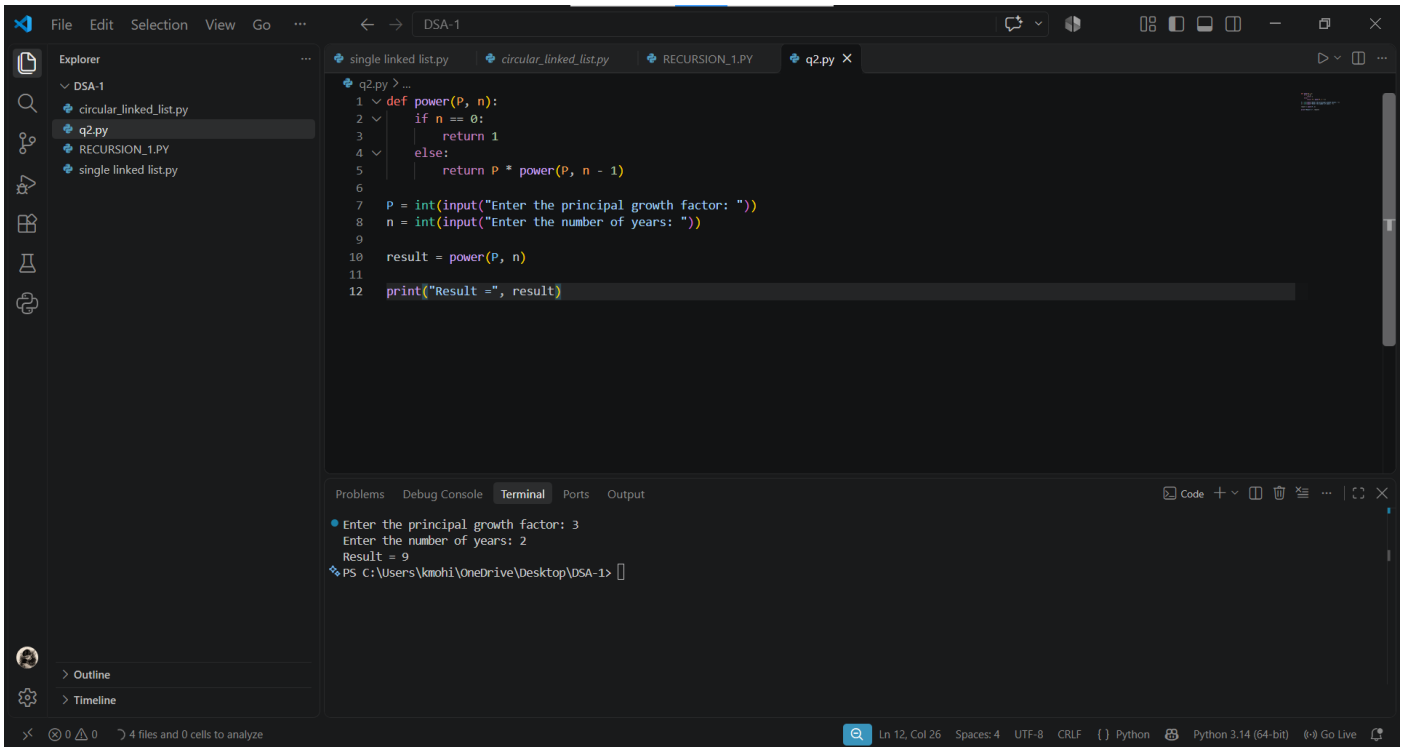
```
def power(P, n):
    if n == 0:
        return 1
    else:
        return P * power(P, n - 1)

P = int(input("Enter the principal growth factor: "))
n = int(input("Enter the number of years: "))

result = power(P, n)

print("Result =", result)
```

Output:



c)A HR department stores employee details in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

Program:

```
def search(emp, eid, i):
    if i == len(emp):
        return -1
    elif emp[i] == eid:
        return i
    else:
        return search(emp, eid, i + 1)


emp = [101, 102, 103, 104, 105]

eid = int(input("Enter Employee ID to search: "))

result = search(emp, eid, 0)

if result == -1:
    print("Employee ID not found")
else:
    print("Employee ID found at index", result)
```

Output:



d)In a logistic department, the no of possible ways to arrange n diff parcels are calculated using the factorial formula(n!). Write a recursive formula to find the factorial of a given number.

Program:

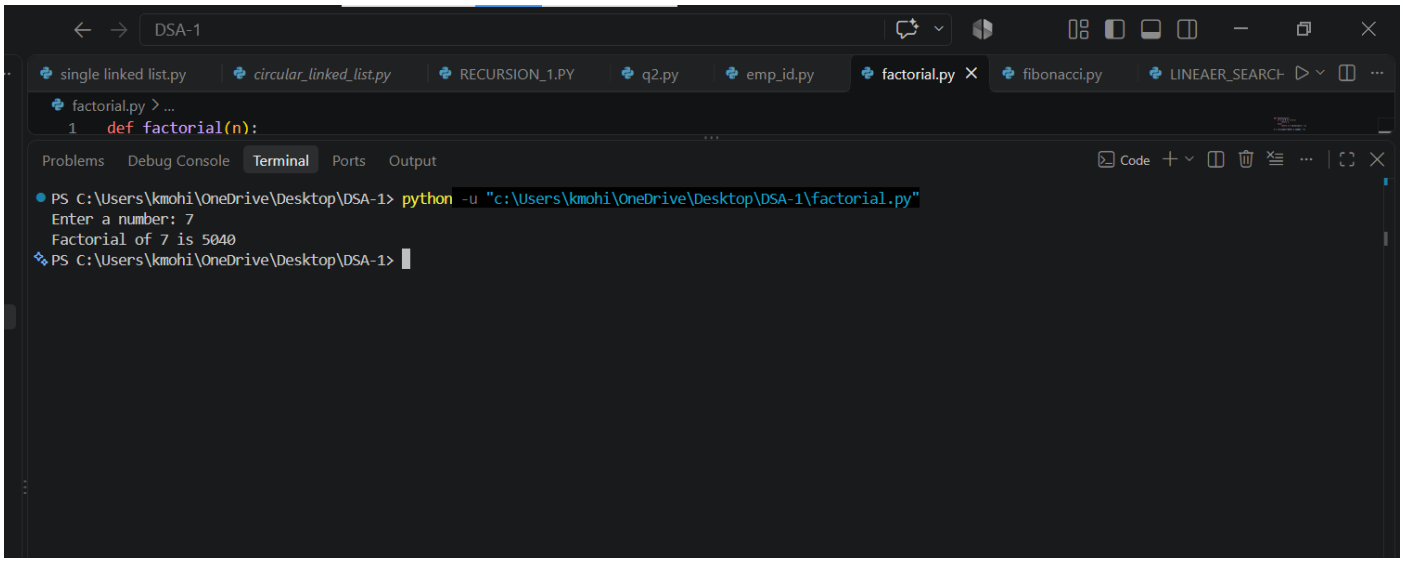
```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

n = int(input("Enter a number: "))

result = factorial(n)

print("Factorial of", n, "is", result)
```

Output:



e)Print first n terms in a fibanocci number series

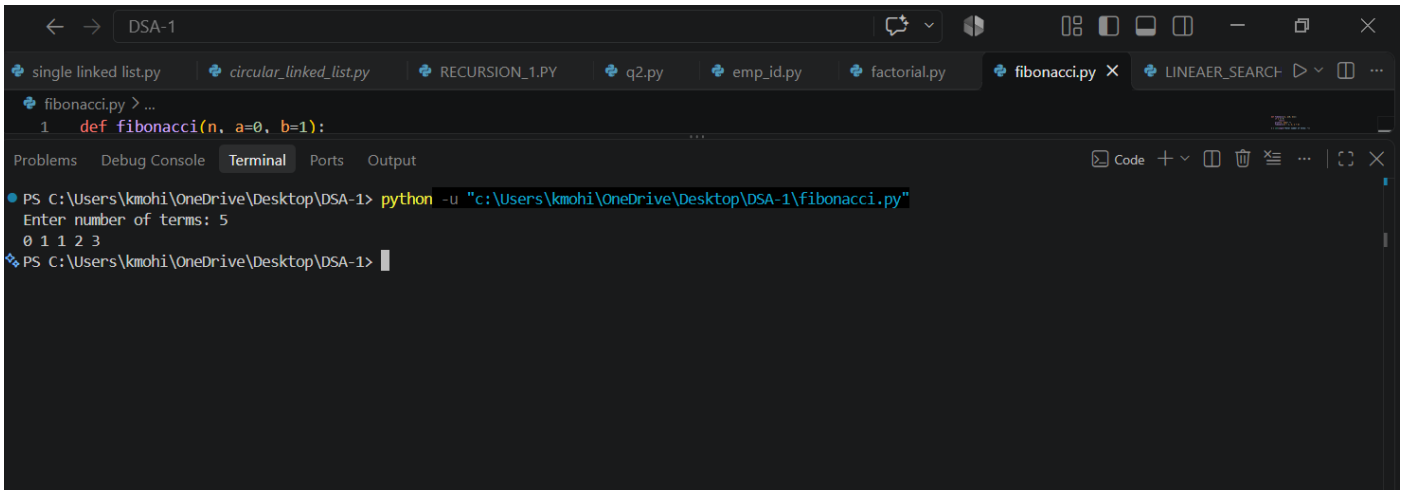
Program:

```
def fibonacci(n, a=0, b=1):
    if n == 0:
        return
    print(a, end=" ")
    fibonacci(n - 1, b, a + b)

n = int(input("Enter number of terms: "))

fibonacci(n)
```

Output:



WEEK 2

Aim:

Write the implementation for the linear search algorithm

Program:

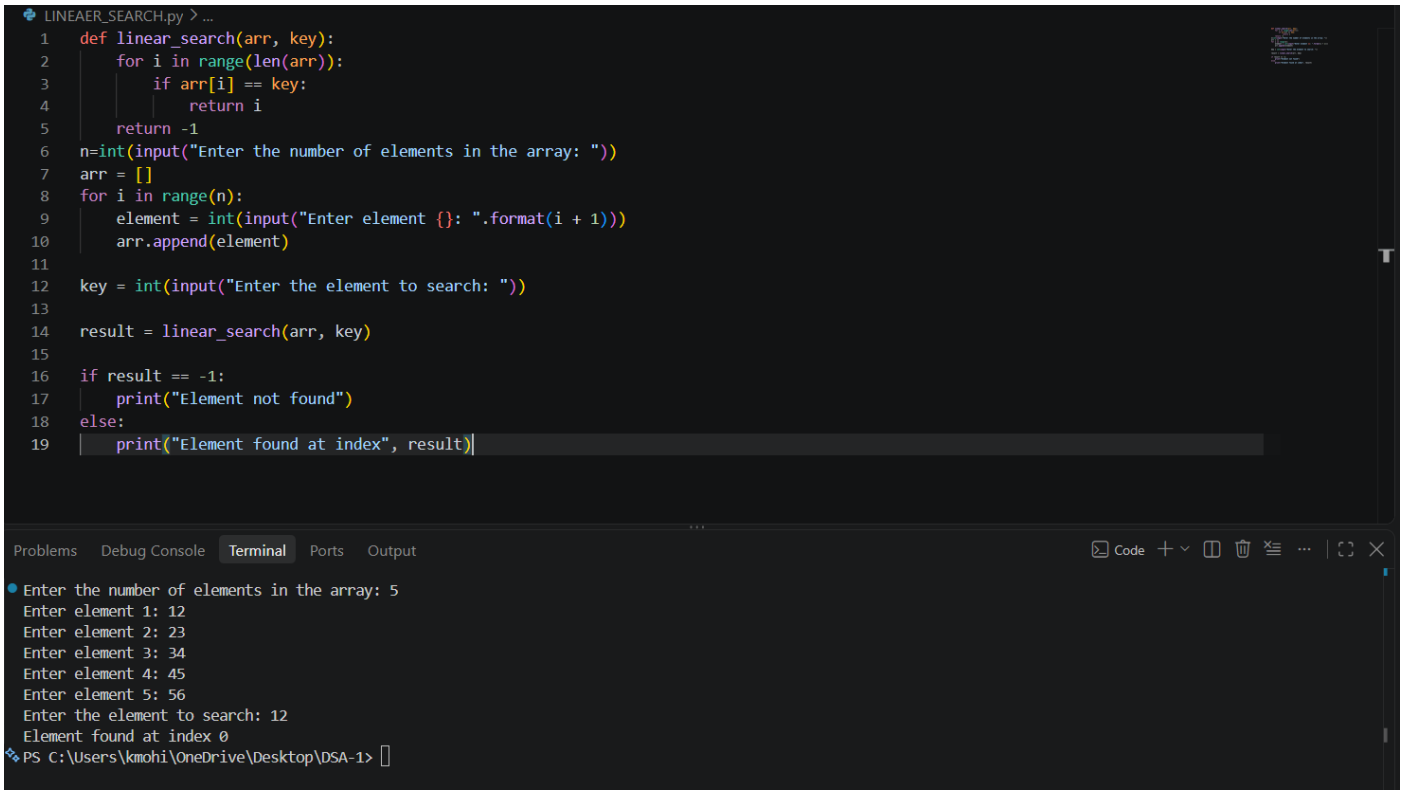
```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1
n=int(input("Enter the number of elements in the array: "))
arr = []
for i in range(n):
    element = int(input("Enter element {}: ".format(i + 1)))
    arr.append(element)

key = int(input("Enter the element to search: "))

result = linear_search(arr, key)

if result == -1:
    print("Element not found")
else:
    print("Element found at index", result)
```

Output:



Aim:

Write the implementation for the binary search algorithm

Program:

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif key < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1

    return -1

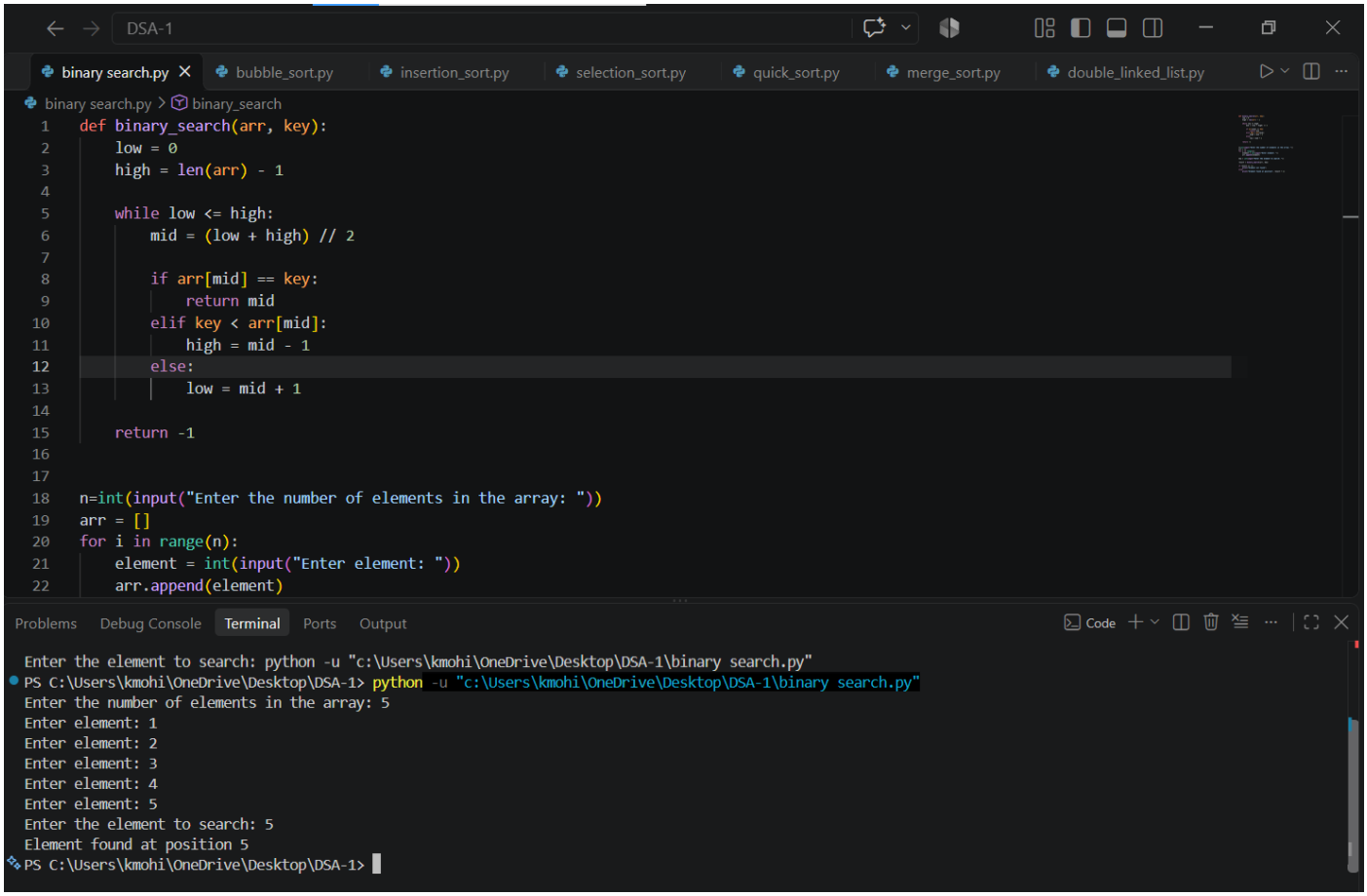

n=int(input("Enter the number of elements in the array: "))
arr = []
for i in range(n):
    element = int(input("Enter element: "))
    arr.append(element)

key = int(input("Enter the element to search: "))

result = binary_search(arr, key)

if result == -1:
    print("Element not found")
else:
    print("Element found at position", result + 1)
```

Output:



WEEK 3

Aim:

Write a python program to implement bubble sort.

Program:

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

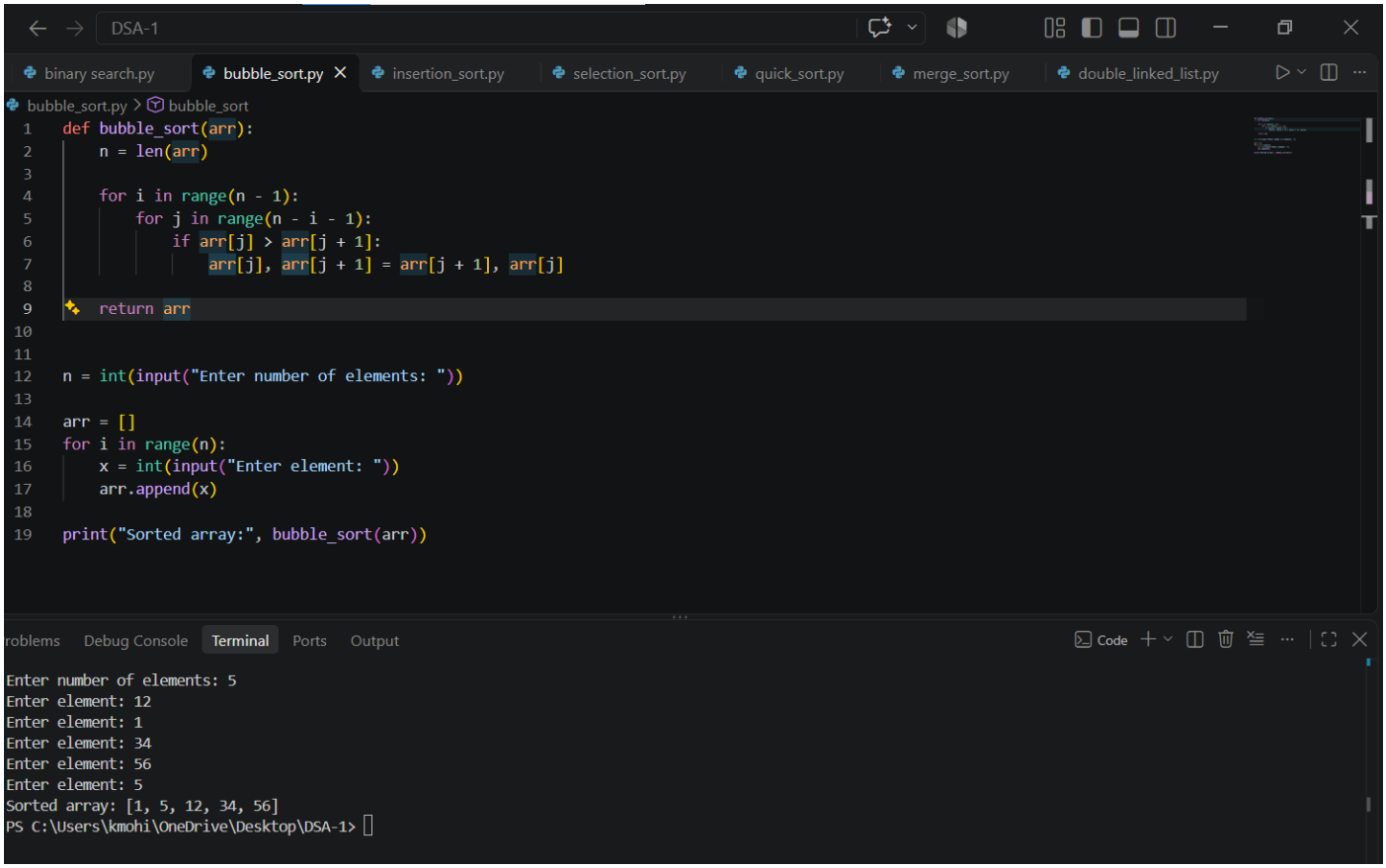
    return arr


n = int(input("Enter number of elements: "))

arr = []
for i in range(n):
    x = int(input("Enter element: "))
    arr.append(x)

print("Sorted array:", bubble_sort(arr))
```

Output:



Aim:

Write a python program to implement insertion sort.

Program:

```
def insertion_sort(arr):
    n = len(arr)

    for i in range(1, n):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j = j - 1

        arr[j + 1] = key

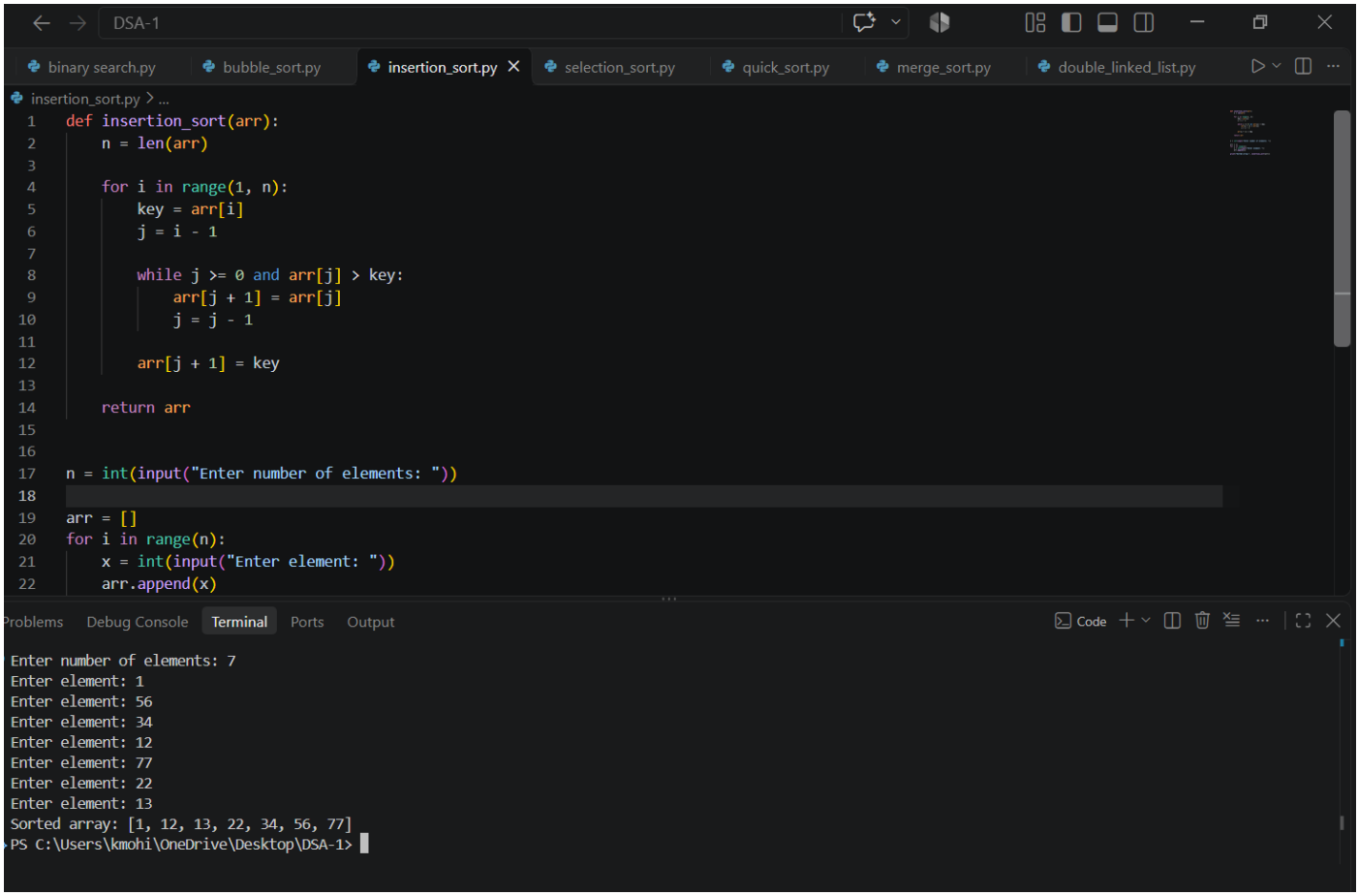
    return arr


n = int(input("Enter number of elements: "))

arr = []
for i in range(n):
    x = int(input("Enter element: "))
    arr.append(x)

print("Sorted array:", insertion_sort(arr))
```

Output:



Aim:

Write a python program to implement selection sort.

Program:

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr

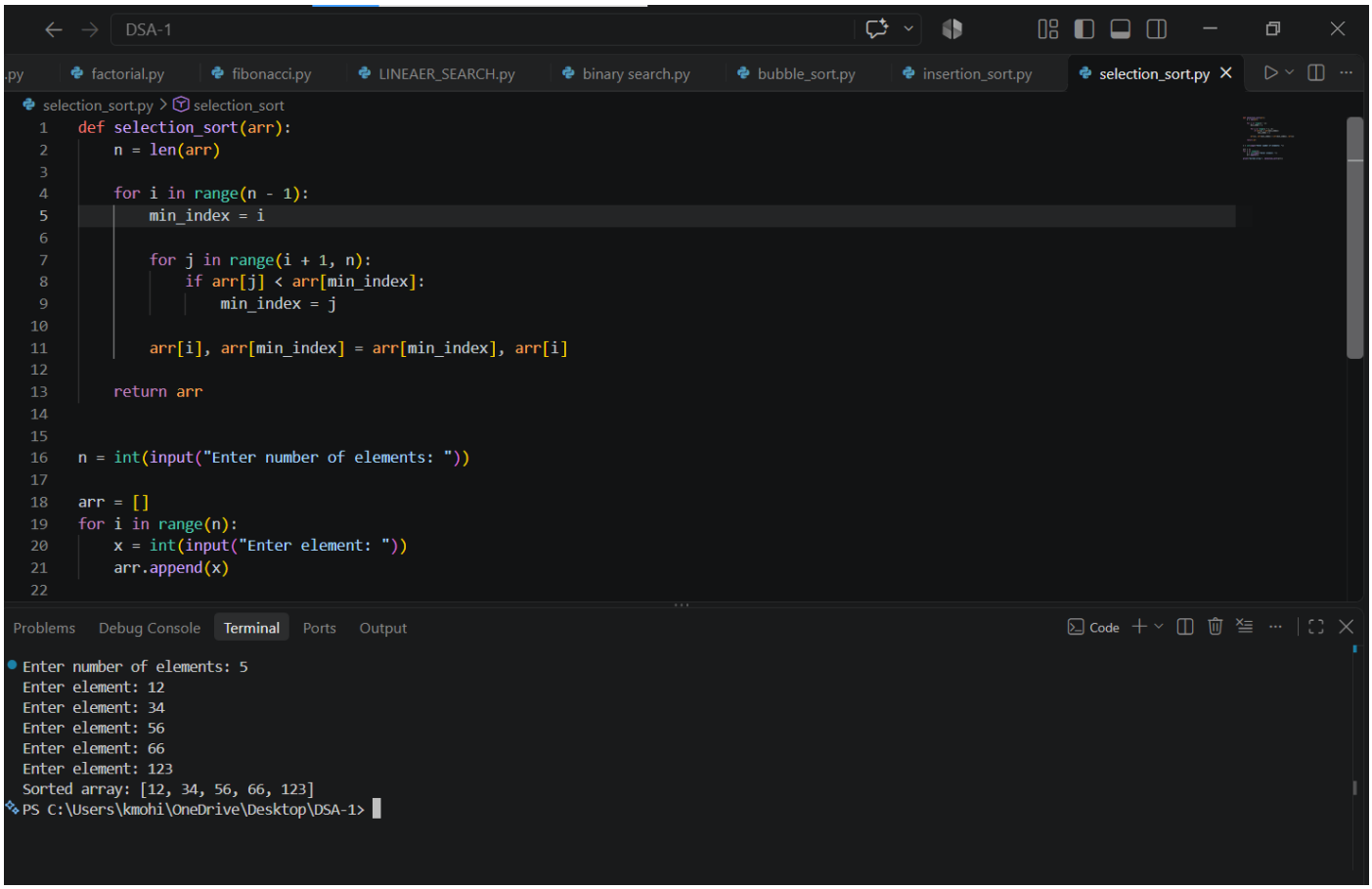

n = int(input("Enter number of elements: "))

arr = []
for i in range(n):
    x = int(input("Enter element: "))
```

```
    arr.append(x)

print("Sorted array:", selection_sort(arr))
```

Output:



WEEK 4

Aim:

Write a python program to implement quick sort.

Program:

```
def quicksort(a, low, high):
    if low < high:
        i, j, pivot = low, high, low

        while i < j:
            while i < len(a) and a[i] <= a[pivot]:
                i += 1
            while a[j] > a[pivot]:
                j -= 1
            if i < j:
                a[i], a[j] = a[j], a[i]

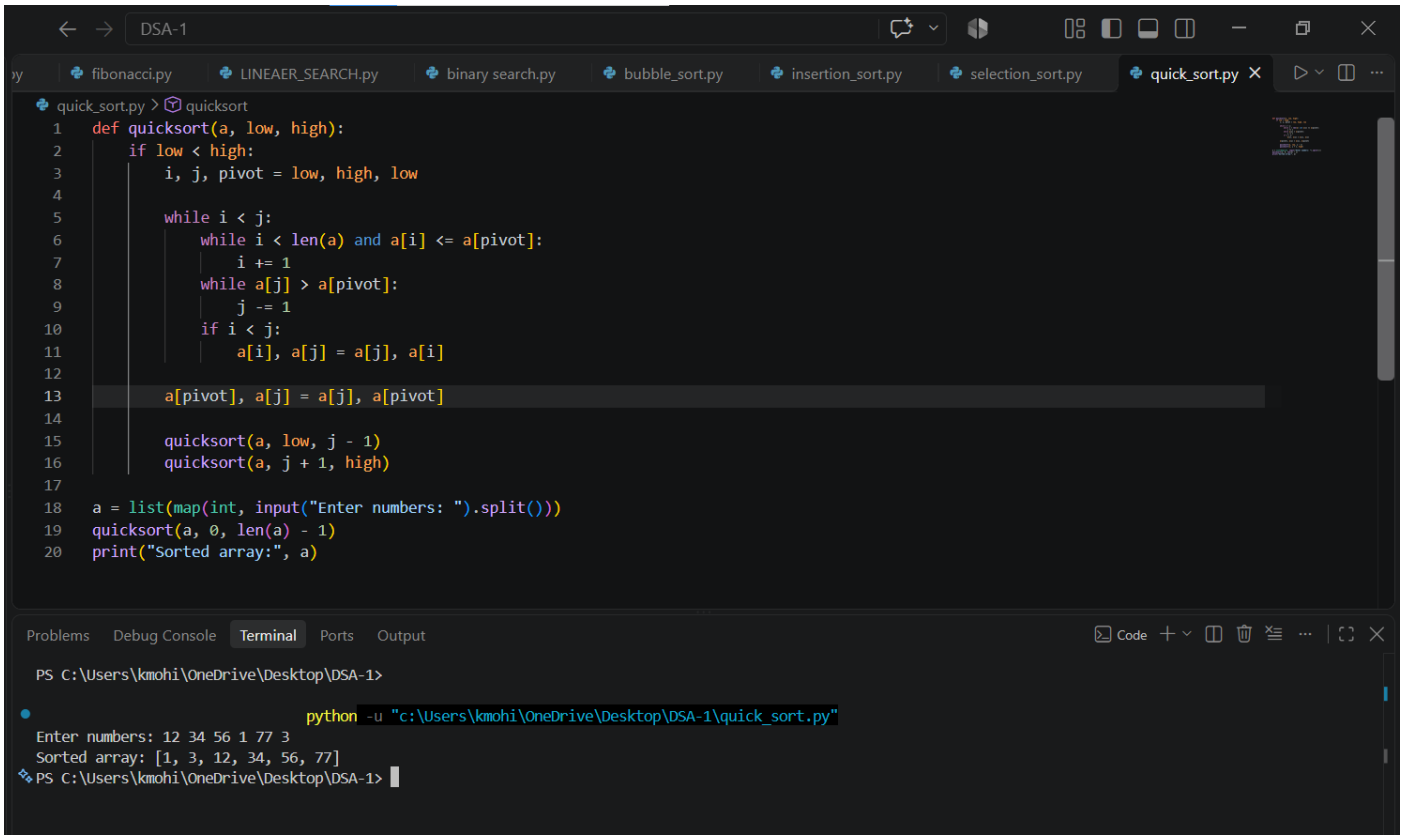
        a[pivot], a[j] = a[j], a[pivot]

        quicksort(a, low, j - 1)
        quicksort(a, j + 1, high)

a = list(map(int, input("Enter numbers: ").split()))
```

```
quicksort(a, 0, len(a) - 1)
print("Sorted array:", a)
```

Output:



Aim:

Write a python program to implement merge sort.

Program:

```
def merge_sort(a):
    if len(a) > 1:
        m = len(a) // 2
        L, R = a[:m], a[m:]

        merge_sort(L)
        merge_sort(R)

        i = j = k = 0

        while i < len(L) and j < len(R):
            if L[i] < R[j]:
                a[k] = L[i]
                i += 1
            else:
                a[k] = R[j]
                j += 1
            k += 1

        while i < len(L):
            a[k] = L[i]
            i += 1
            k += 1
```

```
        while j < len(R):
            a[k] = R[j]
            j += 1
            k += 1

a = list(map(int, input("Enter numbers: ").split()))
merge_sort(a)
print("Sorted array:", a)
```

Output:



WEEK 5

Aim:

Write a python program to implement a Singly Linked List with the below operations.

12) Create Linked List
13) Insert Beginning
14) Insert End
15) Insert at Index
16) Delete by Value
17) Delete First Node
18) Delete Last Node
19) Count number of nodes
20) Sort the data of SLL
21) Search data in SLL
22) Display or Traverse

12)Exit

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class SinglyLinkedList:
    def __init__(self):
        self.head = None
    def create(self):
```

```
        n = int(input("Enter the number of elements: "))
        for i in range(n):
            x = int(input("Enter the value: "))
            new = Node(x)
            if self.head is None:
                self.head = new
            else:
                temp = self.head
                while temp.next:
                    temp = temp.next
                temp.next = new
    def insert_begin(self, data):
        new = Node(data)
        new.next = self.head
        self.head = new
    def insert_end(self, data):
        new = Node(data)
        if self.head is None:
            self.head = new
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new
    def insert_specific(self, index, data):
        if index < 0:
            print("Invalid index")
            return
        if index == 0:
            self.insert_begin(data)
            return

        temp = self.head
        for i in range(index - 1):
            if temp is None:
                print("Invalid index")
                return
            temp = temp.next
        if temp is None:
            print("Invalid index")
            return

        new = Node(data)
        new.next = temp.next
        temp.next = new

    def deleteAtBegin(self):
        if self.head is None:
            print("No data to delete")
        else:
            temp = self.head
            self.head = temp.next
            print("Deleted element is:", temp.data)

    def deleteAtEnd(self):
        if self.head is None:
            print("No data to delete")

        elif self.head.next is None:
            print("Deleted element is:", self.head.data)
            self.head = None

        else:
            temp = self.head
```

```
            temp1 = temp

            while temp.next:
                temp1 = temp
                temp = temp.next

            temp1.next = None
            print("Deleted element is:", temp.data)

    def delete(self, value):
        if self.head is None:
            print("No data to delete")
            return

        temp = self.head

        if temp.data == value:
            self.head = temp.next
            print("Value deleted")
            return

        while temp.next and temp.next.data != value:
            temp = temp.next

        if temp.next is None:
            print("Value not present")
        else:
            temp.next = temp.next.next
            print("Value deleted")
    def display(self):
        if self.head is None:
            print("No data to display")
        else:
            temp = self.head
            while temp:
                print(temp.data, end=" ")
                temp = temp.next
            print()
    def count(self):
        if self.head is None:
            print("No Linked list")
        else:
            c=0
            temp=self.head
            while temp:
                c+=1
                temp=temp.next
            print("Number of nodes:", c)
    def search(self, value):
        if self.head is None:
            print("No data to search")
            return

        temp = self.head
        index = 0

        while temp:
            if temp.data == value:
                print(f"Value found at index: {index}")
                return
            temp = temp.next
            index += 1

        print("Value not found in the list")
```

```
    def sort(self):
        if self.head is None:
            print("No data to sort")
            return

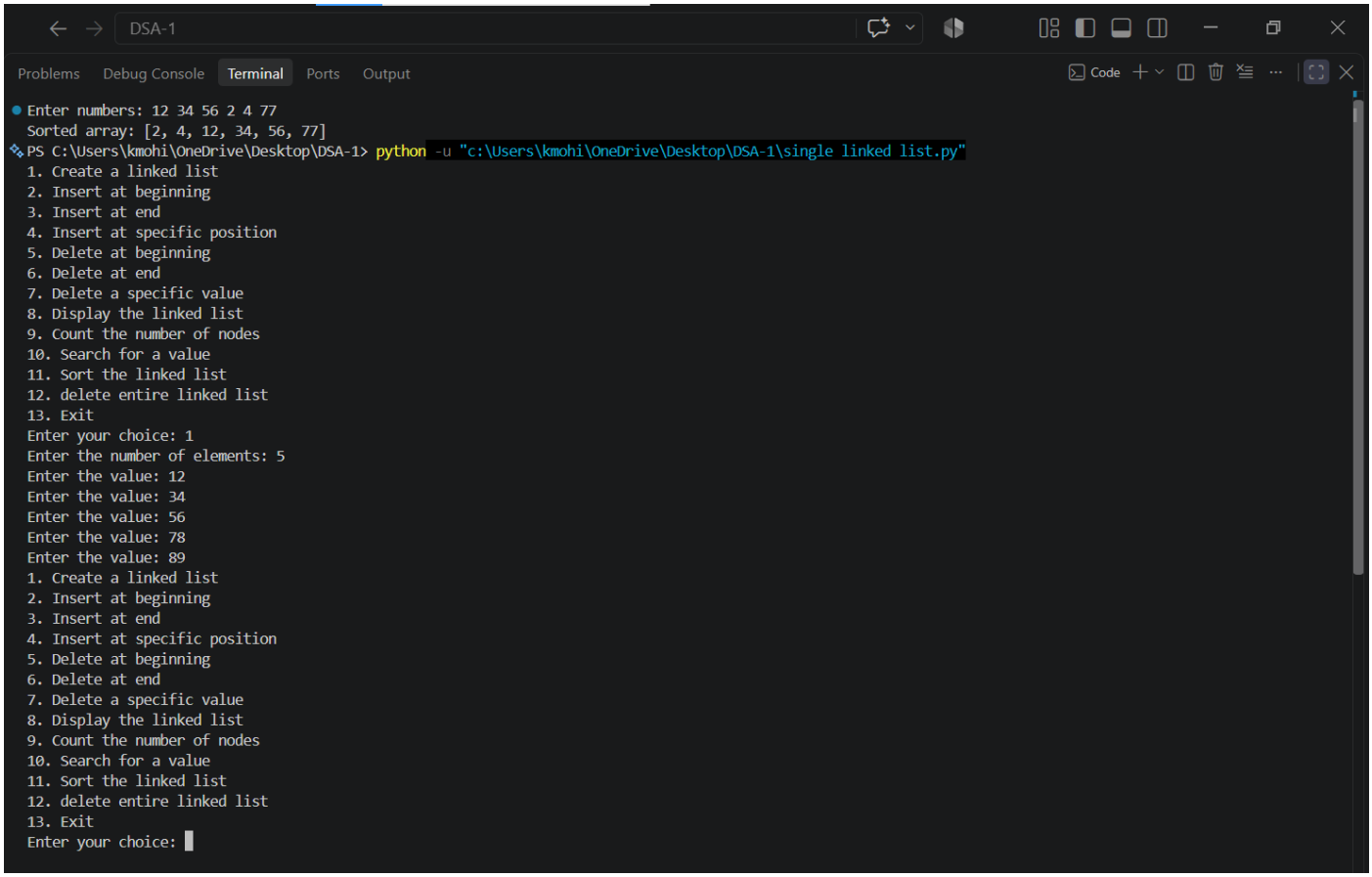
        current = self.head
        while current:
            index = current.next
            while index:
                if current.data > index.data:
                    current.data, index.data = index.data, current.data
                index = index.next
            current = current.next
        print("Linked list sorted")
    def delete_all(self):
        self.head = None
        print("Entire linked list deleted")
sll = SinglyLinkedList()
while True:
    print("1. Create a linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific position")
    print("5. Delete at beginning")
    print("6. Delete at end")
    print("7. Delete a specific value")
    print("8. Display the linked list")
    print("9. Count the number of nodes")
    print("10. Search for a value")
    print("11. Sort the linked list")
    print("12. delete entire linked list")
    print("13. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        sll.create()
    elif choice == 2:
        data = int(input("Enter the value to insert: "))
        sll.insert_begin(data)
    elif choice == 3:
        data = int(input("Enter the value to insert: "))
        sll.insert_end(data)
    elif choice == 4:
        index = int(input("Enter the index to insert: "))
        data = int(input("Enter the value to insert: "))
        sll.insert_specific(index, data)
    elif choice == 5:
        sll.deleteAtBegin()
    elif choice == 6:
        sll.deleteAtEnd()
    elif choice == 7:
        value = int(input("Enter the value to delete: "))
        sll.delete(value)
    elif choice == 8:
        sll.display()
    elif choice == 9:
        sll.count()
    elif choice == 10:
        value = int(input("Enter the value to search: "))
        sll.search(value)
    elif choice == 11:
        sll.sort()
    elif choice == 12:
```

```
        sll.delete_all()
    elif choice == 13:
        break
    else:
        print("Invalid choice, please try again.")
```

Output:



WEEK 6

Aim:

Write a python program to implement a Double Linked List with the below operations.

10) Create Linked List
11) Insert Beginning
12) Insert End
13) Insert at Index
14) Delete by Value
15) Delete First Node
16) Delete Last Node
17) Count number of nodes
18) Display or Traverse

10)Exit

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None

    # Create Linked List
    def create(self):
        n = int(input("Enter the number of elements: "))

        for i in range(n):
            x = int(input("Enter the value: "))
            new = Node(x)

            if self.head is None:
                self.head = new
            else:
                temp = self.head
                while temp.next:
                    temp = temp.next

                temp.next = new
                new.prev = temp

    # Insert at Beginning
    def insert_begin(self, data):
        new = Node(data)

        new.next = self.head

        if self.head is not None:
            self.head.prev = new

        self.head = new

    # Insert at End
    def insert_end(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
        else:
            temp = self.head

            while temp.next:
                temp = temp.next

            temp.next = new
            new.prev = temp
```

```
    # Insert at Specific Index
    def insert_specific(self, index, data):
        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            self.insert_begin(data)
            return

        temp = self.head

        for i in range(index - 1):
            if temp is None:
                print("Invalid index")
                return
            temp = temp.next

        if temp is None:
            print("Invalid index")
            return

        new = Node(data)

        new.next = temp.next
        new.prev = temp

        if temp.next is not None:
            temp.next.prev = new

        temp.next = new

    # Delete First Node
    def deleteAtBegin(self):
        if self.head is None:
            print("No data to delete")
        else:
            temp = self.head
            self.head = temp.next

            if self.head is not None:
                self.head.prev = None

            print("Deleted element is:", temp.data)

    # Delete Last Node
    def deleteAtEnd(self):
        if self.head is None:
            print("No data to delete")

        elif self.head.next is None:
            print("Deleted element is:", self.head.data)
            self.head = None

        else:
```

```
            temp = self.head

            while temp.next:
                temp = temp.next

            temp1 = temp.prev
            temp1.next = None

            print("Deleted element is:", temp.data)

    # Delete by Value
    def delete(self, value):
        if self.head is None:
            print("No data to delete")
            return

        temp = self.head

        if temp.data == value:
            self.head = temp.next

            if self.head is not None:
                self.head.prev = None

            print("Value deleted")
            return

        while temp and temp.data != value:
            temp = temp.next

        if temp is None:
            print("Value not present")
        else:
            temp.prev.next = temp.next

            if temp.next is not None:
                temp.next.prev = temp.prev

            print("Value deleted")

    # Count Number of Nodes
    def count(self):
        if self.head is None:
            print("No Linked list")
        else:
            c = 0
            temp = self.head

            while temp:
                c += 1
                temp = temp.next

            print("Number of nodes:", c)

    # Display / Traverse
    def display(self):
```

```
        if self.head is None:
            print("No data to display")
        else:
            temp = self.head

            while temp:
                print(temp.data, end=" ")
                temp = temp.next

            print()


# Main Program
dll = DoublyLinkedList()

while True:
    print("\n1. Create a linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific position")
    print("5. Delete a specific value")
    print("6. Delete at beginning")
    print("7. Delete at end")
    print("8. Count the number of nodes")
    print("9. Display the linked list")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        dll.create()

    elif choice == 2:
        data = int(input("Enter the value to insert: "))
        dll.insert_begin(data)

    elif choice == 3:
        data = int(input("Enter the value to insert: "))
        dll.insert_end(data)

    elif choice == 4:
        index = int(input("Enter the index to insert: "))
        data = int(input("Enter the value to insert: "))
        dll.insert_specific(index, data)

    elif choice == 5:
        value = int(input("Enter the value to delete: "))
        dll.delete(value)

    elif choice == 6:
        dll.deleteAtBegin()

    elif choice == 7:
        dll.deleteAtEnd()

    elif choice == 8:
```

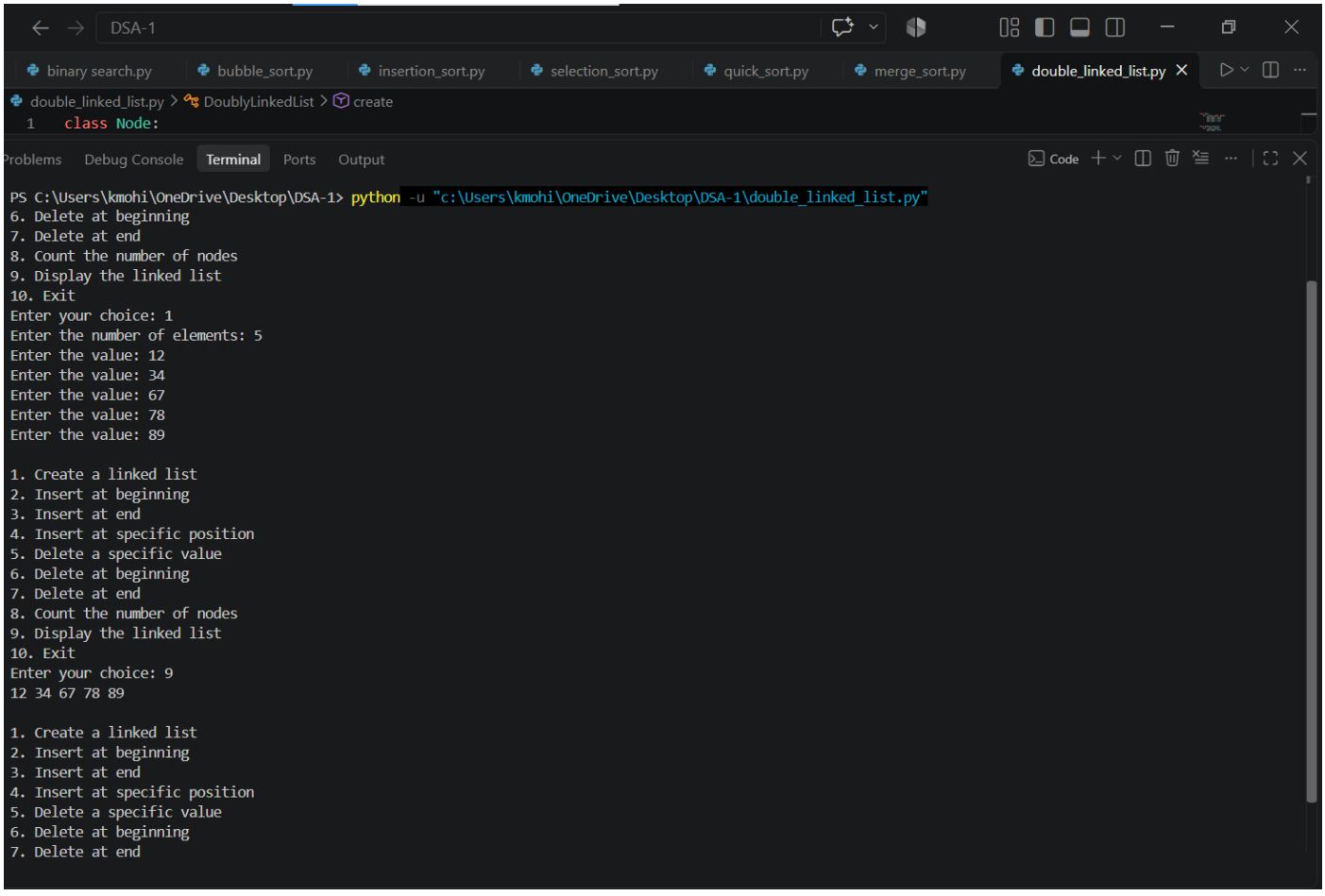
```
        dll.count()

    elif choice == 9:
        dll.display()

    elif choice == 10:
        break

    else:
        print("Invalid choice, please try again.")
```

Output:



WEEK 7

Aim:

Write a python program to implement stack operations such as push, pop, peek, display using an array.

Program:

```
stack = []
top = -1
n = int(input("Enter stack size: "))

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    ch = int(input("Enter your choice: "))

    if ch == 1:
        if top == n - 1:
            print("Stack Overflow")
        else:
            x = int(input("Enter element: "))
            stack.append(x)
            top += 1

    elif ch == 2:
        if top == -1:
            print("Stack Underflow")
        else:
            print("Deleted element:", stack.pop())
            top -= 1

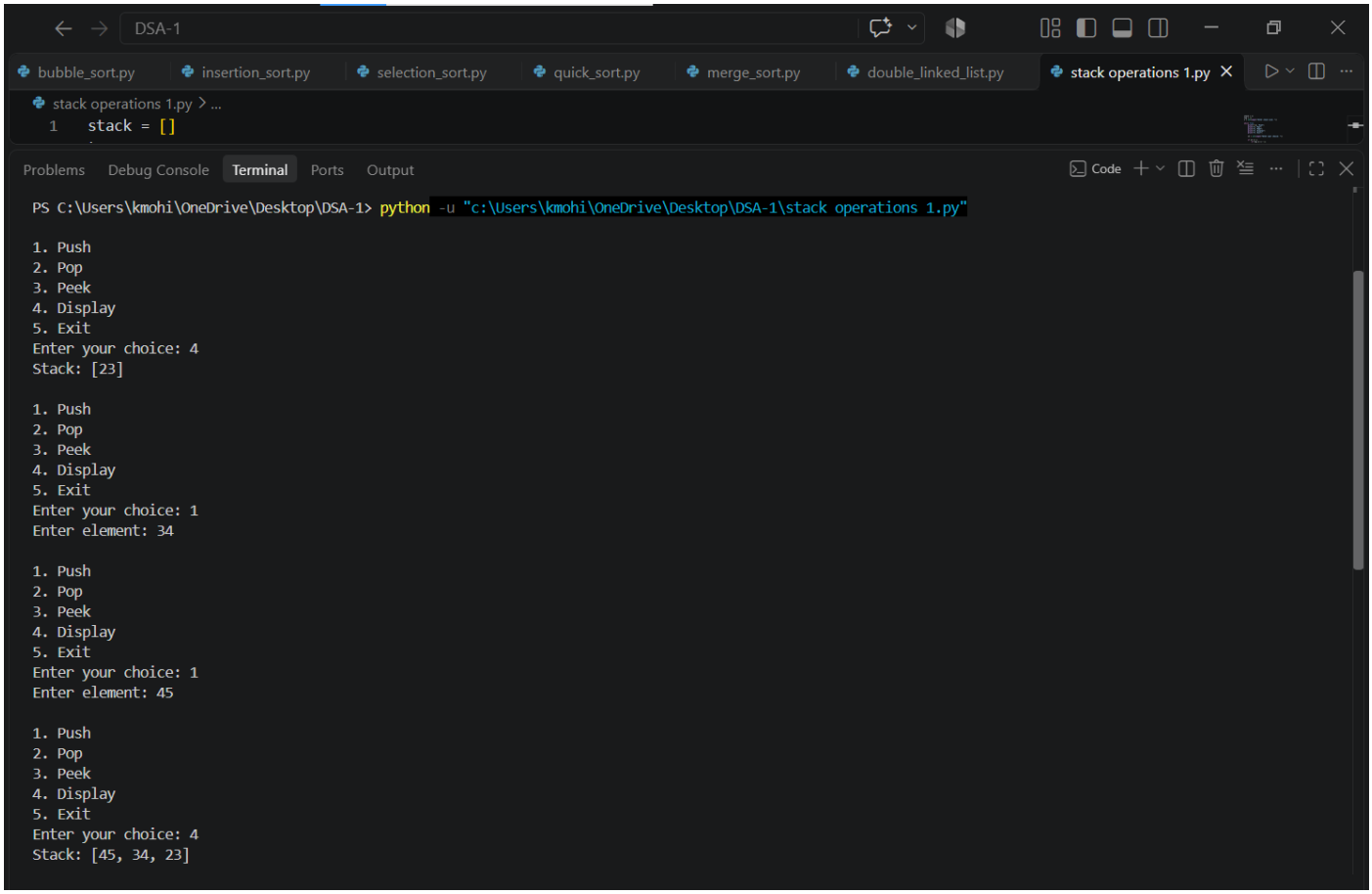
    elif ch == 3:
        if top == -1:
            print("Stack is Empty")
        else:
            print("Top element:", stack[top])

    elif ch == 4:
        if top == -1:
            print("Stack is Empty")
        else:
            print("Stack:", stack[::-1])

    elif ch == 5:
        break

    else:
        print("Invalid Choice")
```

Output:



Aim:

Write a python program to implement stack operations such as push, pop, peek, display using a linked list.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

top = None

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    ch = int(input("Enter your choice: "))

    if ch == 1:
        x = int(input("Enter element: "))
        new = Node(x)
        new.next = top
        top = new

    elif ch == 2:
        if top is None:
```

```
            print("Stack Underflow")
        else:
            print("Deleted element:", top.data)
            top = top.next

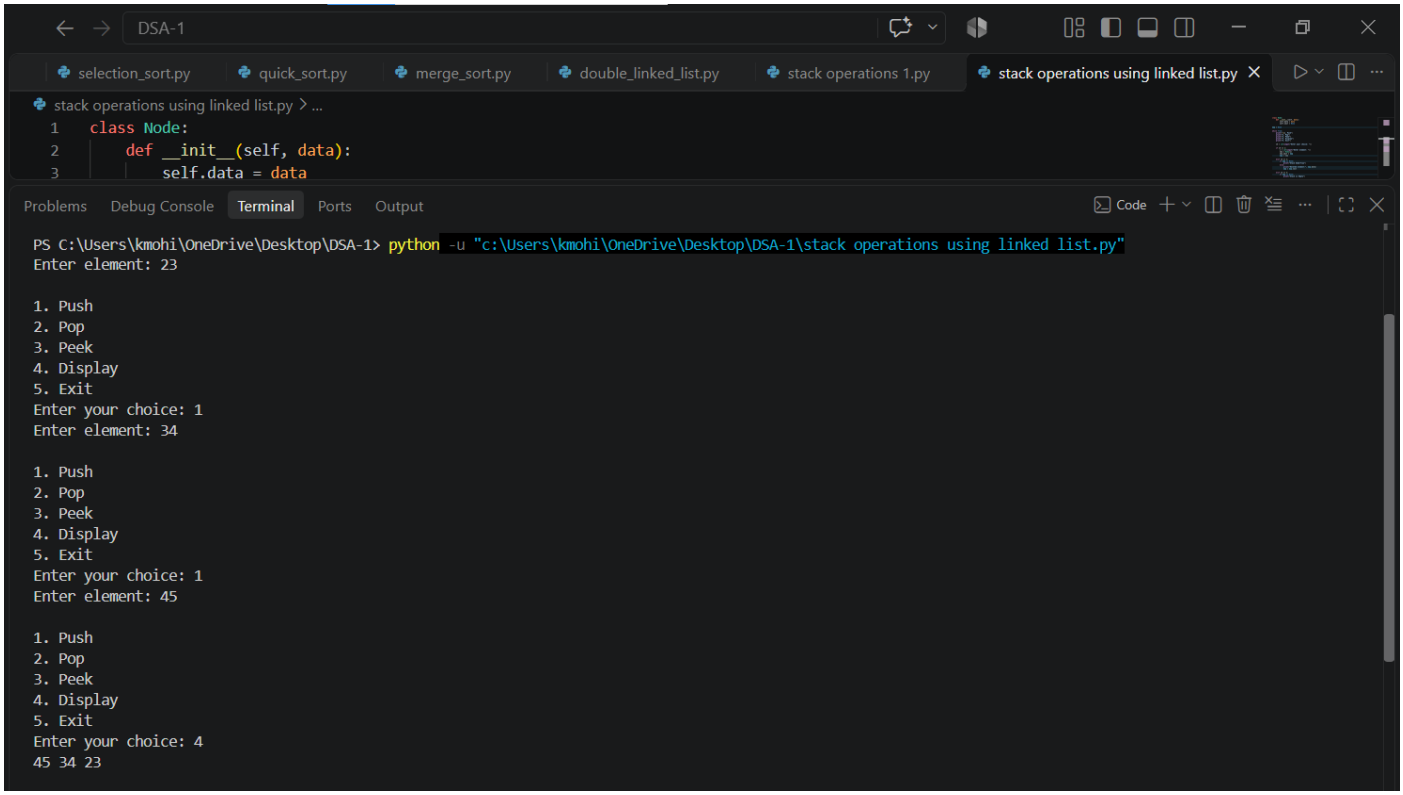
    elif ch == 3:
        if top is None:
            print("Stack is Empty")
        else:
            print("Top element:", top.data)

    elif ch == 4:
        if top is None:
            print("Stack is Empty")
        else:
            temp = top
            while temp:
                print(temp.data, end=" ")
                temp = temp.next
            print()

    elif ch == 5:
        break

    else:
        print("Invalid Choice")
```

Output:



WEEK 8

AIM:

Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

Program of queue using array:

```
queue = []
front = rear = -1
n = int(input("Enter queue size: "))

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    ch = int(input("Enter your choice: "))

    if ch == 1:
        if rear == n - 1:
            print("Queue Overflow")
        else:
            x = int(input("Enter element: "))
            queue.append(x)
            if front == -1:
                front = 0
            rear += 1

    elif ch == 2:
        if front == -1 or front > rear:
            print("Queue Underflow")
        else:
            print("Deleted element:", queue[front])
            front += 1

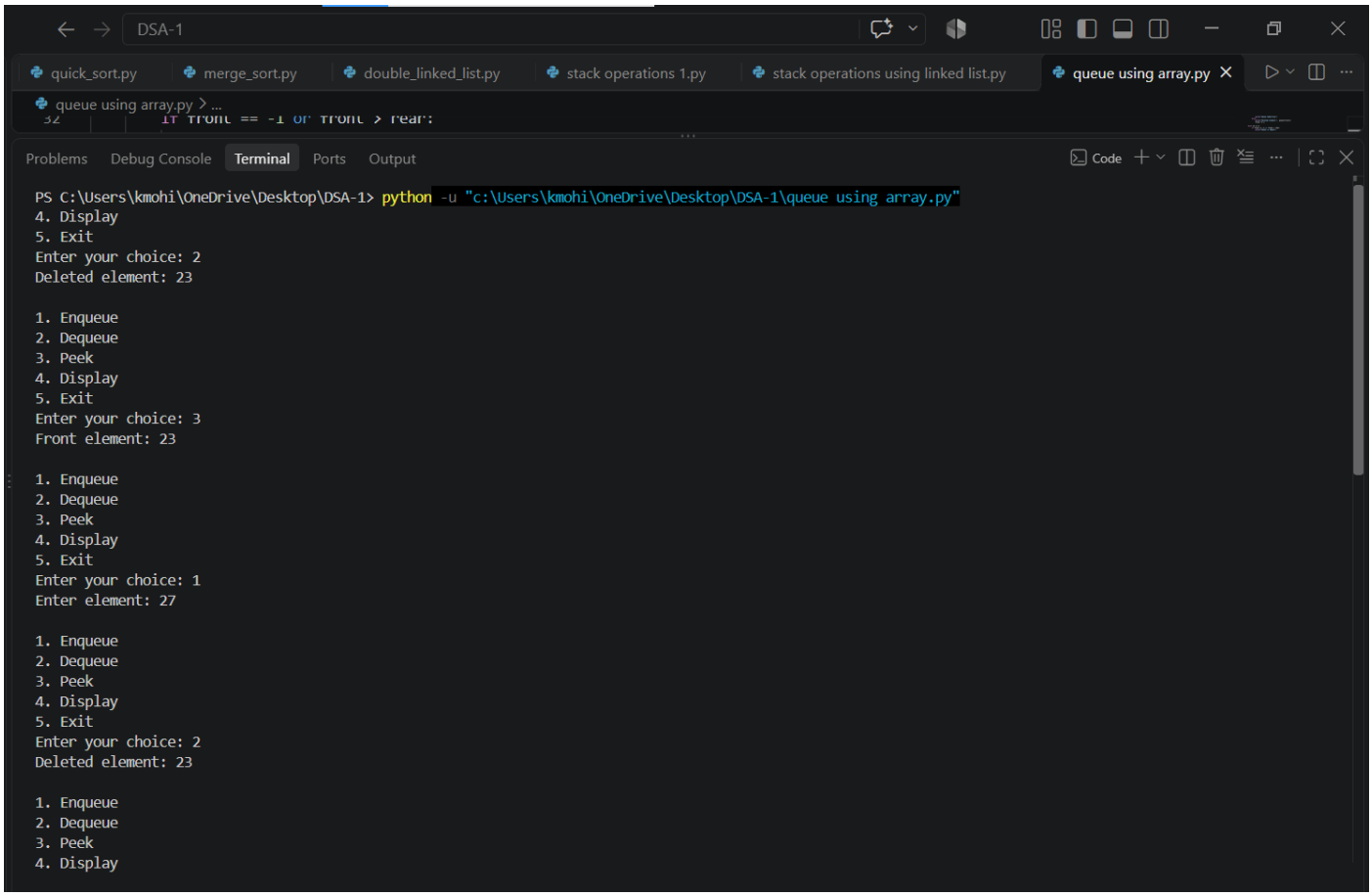
    elif ch == 3:
        if front == -1 or front > rear:
            print("Queue is Empty")
        else:
            print("Front element:", queue[front])

    elif ch == 4:
        if front == -1 or front > rear:
            print("Queue is Empty")
        else:
            print("Queue:", queue[front:rear+1])

    elif ch == 5:
        break

    else:
        print("Invalid Choice")
```

Output:



Program using linked list:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

front = rear = None

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    ch = int(input("Enter your choice: "))

    if ch == 1:
        x = int(input("Enter element: "))
        new = Node(x)

        if front is None:
            front = rear = new
        else:
            rear.next = new
            rear = new

    elif ch == 2:
        if front is None:
```

```
            print("Queue Underflow")
        else:
            print("Deleted element:", front.data)
            front = front.next
            if front is None:
                rear = None

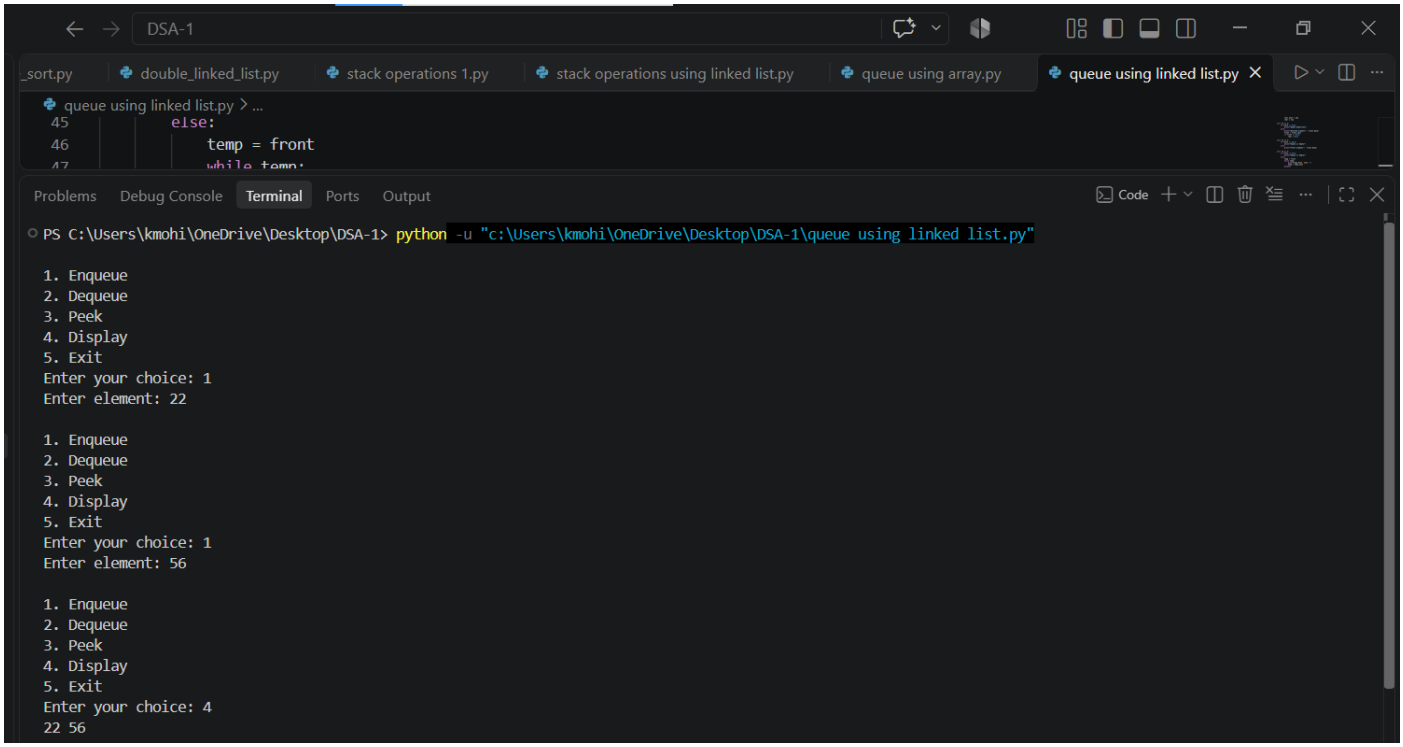
    elif ch == 3:
        if front is None:
            print("Queue is Empty")
        else:
            print("Front element:", front.data)

    elif ch == 4:
        if front is None:
            print("Queue is Empty")
        else:
            temp = front
            while temp:
                print(temp.data, end=" ")
                temp = temp.next
            print()

    elif ch == 5:
        break

    else:
        print("Invalid Choice")
```

Output:



Aim:

Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

Program using array:

```
queue = []
front = rear = -1
n = int(input("Enter queue size: "))

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    ch = int(input("Enter your choice: "))

    if ch == 1:
        if (rear + 1) % n == front:
            print("Queue Overflow")
        else:
            x = int(input("Enter element: "))
            if front == -1:
                front = 0
            rear = (rear + 1) % n
            if len(queue) < n:
                queue.append(None)
            queue[rear] = x

    elif ch == 2:
        if front == -1:
            print("Queue Underflow")
        else:
            print("Deleted element:", queue[front])
            if front == rear:
                front = rear = -1
            else:
                front = (front + 1) % n

    elif ch == 3:
        if front == -1:
            print("Queue is Empty")
        else:
            print("Front element:", queue[front])

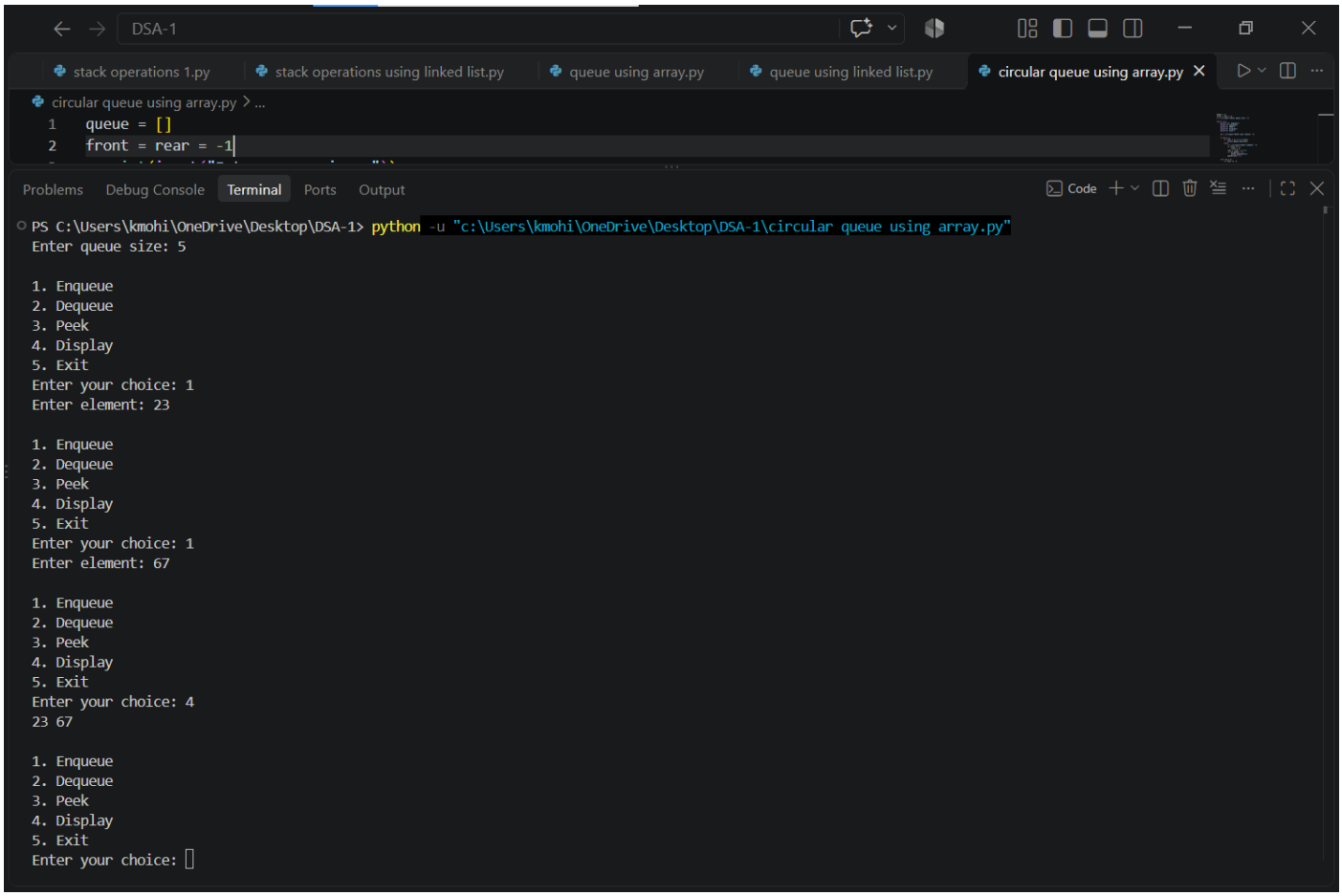
    elif ch == 4:
        if front == -1:
            print("Queue is Empty")
        else:
            i = front
            while True:
```

```
                print(queue[i], end=" ")
                if i == rear:
                    break
                i = (i + 1) % n
            print()

    elif ch == 5:
        break

    else:
        print("Invalid Choice")
```

Output:



Program using linked list:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

front = rear = None

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
```

```
    print("5. Exit")

    ch = int(input("Enter your choice: "))

    if ch == 1:
        x = int(input("Enter element: "))
        new = Node(x)

        if front is None:
            front = rear = new
            new.next = front
        else:
            rear.next = new
            rear = new
            rear.next = front

    elif ch == 2:
        if front is None:
            print("Queue Underflow")
        else:
            print("Deleted element:", front.data)

            if front == rear:
                front = rear = None
            else:
                front = front.next
                rear.next = front

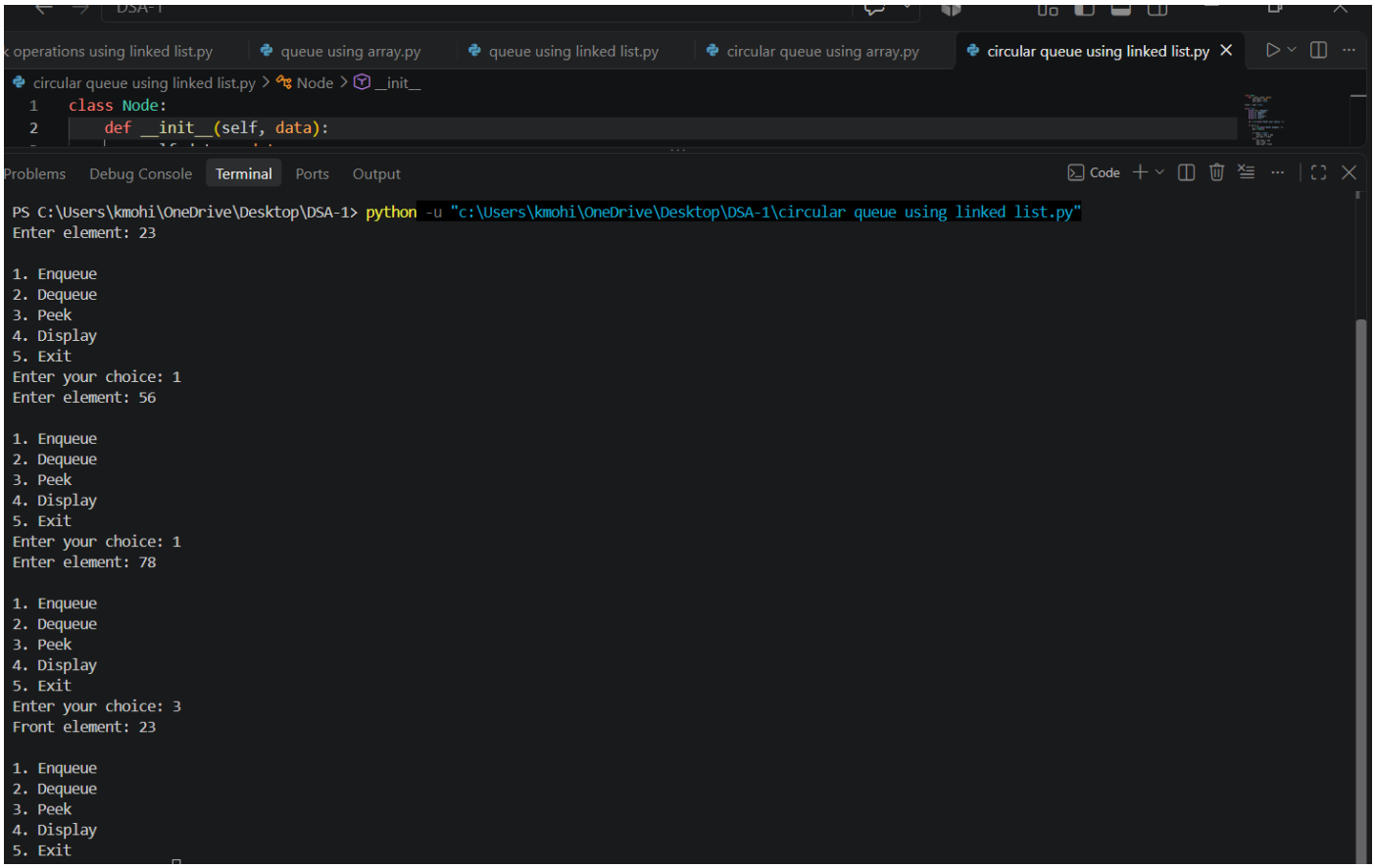
    elif ch == 3:
        if front is None:
            print("Queue is Empty")
        else:
            print("Front element:", front.data)

    elif ch == 4:
        if front is None:
            print("Queue is Empty")
        else:
            temp = front
            while True:
                print(temp.data, end=" ")
                temp = temp.next
                if temp == front:
                    break
            print()

    elif ch == 5:
        break

    else:
        print("Invalid Choice")
```

Output:



WEEK 9

Aim:

Write a python program to implement a Binary Search Tree (BST) and perform the following operations.
1. Insertion
2. Deletion
3. Tree Traversals – in-order, pre-order and post-order
4. Count Leaf Nodes
5. Count Internal Nodes
6. Height of the Tree

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    else:
        print("Duplicate value is not allowed.")
    return root
```

```python
def inorder(root):
    if root is None:
        return
    inorder(root.left)
    print(root.data, end=" ")
    inorder(root.right)

def preorder(root):
    if root is None:
        return
    print(root.data, end=" ")
    preorder(root.left)
    preorder(root.right)

def postorder(root):
    if root is None:
        return
    postorder(root.left)
    postorder(root.right)
    print(root.data, end=" ")

def find_min(root):
    if root is None:
        return None
    while root.left is not None:
        root = root.left
    return root

def find_max(root):
    if root is None:
        return None
    while root.right is not None:
        root = root.right
    return root

def search(root, data):
    if root is None:
        return None
    if data < root.data:
        return search(root.left, data)
    elif data > root.data:
        return search(root.right, data)
    else:
        return root

def count_nodes(root):
    if root is None:
        return 0
    return count_nodes(root.left) + count_nodes(root.right) + 1

def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaves(root.left) + count_leaves(root.right)

def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left) + count_internal(root.right) + 1

def height(root):
    if root is None:
        return -1
    left_height = height(root.left)
    right_height = height(root.right)
    return max(left_height, right_height) + 1


def delete(root, x):
    if root is None:
        print("Element not found.")
        return None
    if x < root.data:
        root.left = delete(root.left, x)
```

```
    elif x > root.data:
        root.right = delete(root.right, x)
    else:
        # Case 1: Node has no children
        if root.left is None and root.right is None:
            root = None
        # Case 2: Node has only right child
        elif root.left is None:
            root = root.right
        # Case 3: Node has only left child
        elif root.right is None:
            root = root.left
        # Case 4: Node has two children
        else:
            successor = find_min(root.right)
            root.data = successor.data
            root.right = delete(root.right, successor.data)
    return root

def print_level(root, level):
    if root is None:
        return
    if level == 0:
        print(root.data, end=" ")
    else:
        print_level(root.left, level - 1)
        print_level(root.right, level - 1)


def level_order(root):
    h = height(root)
    for level in range(h+1):
        print_level(root, level)

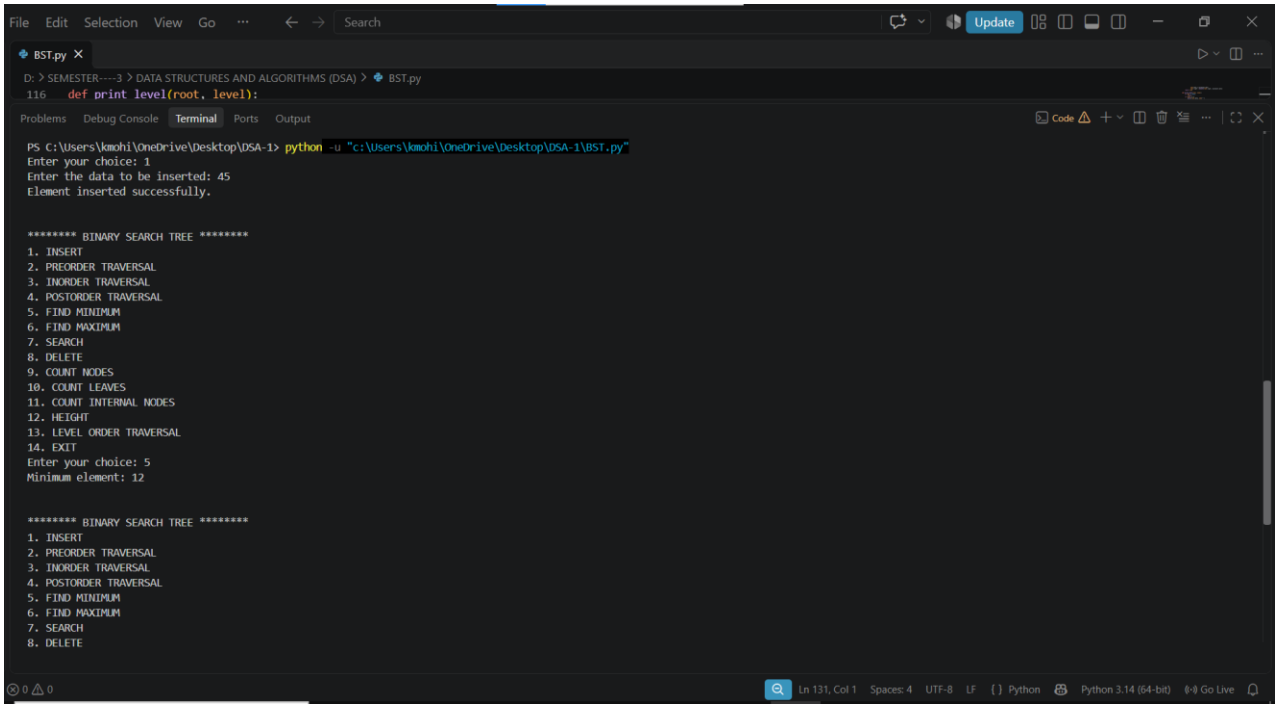

# ====================================================
# MAIN PROGRAM
# ====================================================

root = None
while True:
    print("\n\n******** BINARY SEARCH TREE ********")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. FIND MINIMUM")
    print("6. FIND MAXIMUM")
    print("7. SEARCH")
    print("8. DELETE")
    print("9. COUNT NODES")
    print("10. COUNT LEAVES")
    print("11. COUNT INTERNAL NODES")
    print("12. HEIGHT")
    print("13. LEVEL ORDER TRAVERSAL")
    print("14. EXIT")

    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter the data to be inserted: "))
        root = insert(root, data)
        print("Element inserted successfully.")
    elif choice == 2:
        print("Preorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            preorder(root)
            print()
    elif choice == 3:
        print("Inorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            inorder(root)
            print()
    elif choice == 4:
        print("Postorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
```

```
        else:
            postorder(root)
            print()
    elif choice == 5:
        node = find_min(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Minimum element:", node.data)
    elif choice == 6:
        node = find_max(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Maximum element:", node.data)
    elif choice == 7:
        data = int(input("Enter the data to be searched: "))
        node = search(root, data)
        if node is None:
            print("Element", data, "is not found.")
        else:
            print("Element", data, "is found.")
    elif choice == 8:
        data = int(input("Enter the data to be deleted: "))
        root = delete(root, data)
    elif choice == 9:
        print("Number of nodes:", count_nodes(root))
    elif choice == 10:
        print("Number of leaf nodes:", count_leaves(root))
    elif choice == 11:
        print("Number of internal nodes:", count_internal(root))
    elif choice == 12:
        print("Height of BST:", height(root))
    elif choice == 13:
        print("Level Order Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            level_order(root)
            print()
    elif choice == 14:
        print("Program terminated.")
        break
    else:
        print("Invalid choice. Please try again.")
```

Output:



WEEK 10

Aim:

Write a menu driven python program to implement an AVL Tree and perform the following operations.
1. Insertion
2. Deletion
3. Tree Traversals
4. Count Leaf Nodes
5. Count Internal Nodes
6. Height of the Tree

Program:

```
class Node:
    def __init__(self, x):
        self.left = None
        self.data = x
        self.right = None
        self.h = 0

#AVL TREE Program
def height(root):
    if root is None:
        return -1
    else:
        return root.h

# LL Rotation
def LL(x):
    y = x.left
    x.left = y.right
    y.right = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y

# RR Rotation
def RR(x):
    y = x.right
    x.right = y.left
    y.left = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y

# LR Rotation
def LR(k3):
    k3.left = RR(k3.left)
    return LL(k3)

# RL Rotation
def RL(k3):
    k3.right = LL(k3.right)
    return RR(k3)

# Inorder Traversal
def inorder(root):
    if root is not None:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)


# Preorder Traversal
def preorder(root):
    if root is not None:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)
```

```
# Postorder Traversal
def postorder(root):
    if root is not None:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")

# Insert a node into AVL Tree
def insert(root, x):
    if root is None:
        return Node(x)

    if x < root.data:
        root.left = insert(root.left, x)
        if height(root.left) - height(root.right) == 2:
            if x < root.left.data:
                root = LL(root)
            else:
                root = LR(root)

    elif x > root.data:
        root.right = insert(root.right, x)
        if height(root.right) - height(root.left) == 2:
            if x > root.right.data:
                root = RR(root)
            else:
                root = RL(root)
    else:
        print("Duplicates not allowed")
    root.h = max(height(root.left),height(root.right)) + 1
    return root


# Delete a node from AVL Tree
def delete(root, x):
    # Element not found
    if root is None:
        return None

    # Search in left subtree
    if x < root.data:
        root.left = delete(root.left, x)
        if height(root.left) - height(root.right) == 2:
            if x < root.left.data:
                root = LL(root)
            else:
                root = LR(root)

    # Search in right subtree
    elif x > root.data:
        root.right = delete(root.right, x)
        if height(root.right) - height(root.left) == 2:
            if x > root.right.data:
                root = RR(root)
            else:
                root = RL(root)

    # When target found
    else:
        if root.left is None and root.right is None:
            root = None
        elif root.right is None:
            root = root.left
        elif root.left is None:
            root = root.right
        else:
            successor = root.right   # Find inorder successor
            while successor.left is not None:
                successor = successor.left
            root.data = successor.data   #Copy successor data
            root.right = delete(root.right, successor.data)     # Delete the inorder successor

        root.h = max(height(root.left), height(root.right)) + 1
    return root

# ------------------------------
# Main Program
# ------------------------------

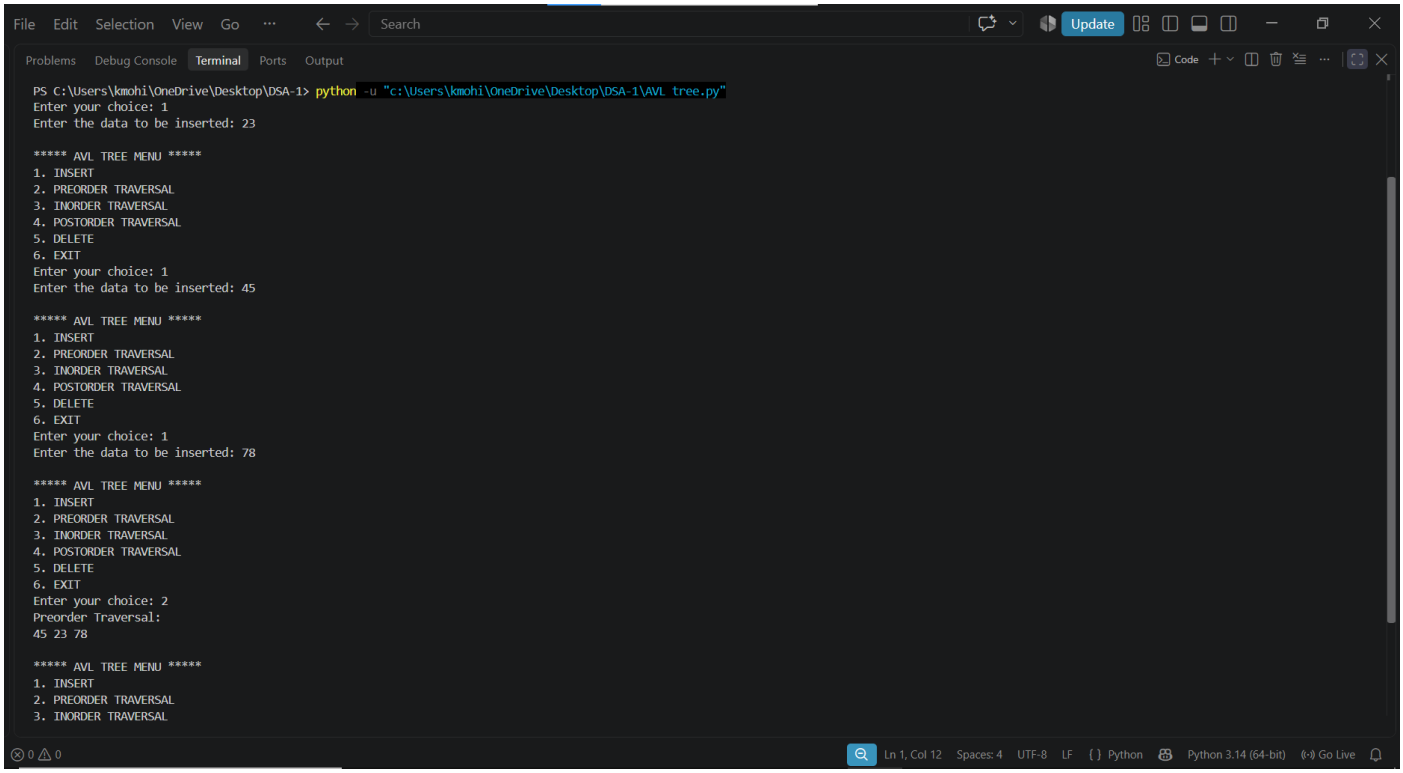
root = None
```

```
while True:

    print("\n***** AVL TREE MENU *****")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. DELETE")
    print("6. EXIT")

    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter the data to be inserted: "))
        root = insert(root, data)
    elif choice == 2:
        print("Preorder Traversal:")
        preorder(root)
        print()
    elif choice == 3:
        print("Inorder Traversal:")
        inorder(root)
        print()
    elif choice == 4:
        print("Postorder Traversal:")
        postorder(root)
        print()
    elif choice == 5:
        data = int(input("Enter the data to be deleted: "))
        root = delete(root, data)
    elif choice == 6:
        print("Program terminated.")
        break
    else:
        print("Invalid choice!")
```

Output:



WEEK 11

Aim:

Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations:

5. Insertion of an element into the Heap.
6. Deletion of the root element from the Heap.
7. Display the array representation of the Heap.
4. Exit the program

Program:

```
def insert(heap, value):
    heap.append(value)
    i = len(heap) - 1
    while i > 0:
        parent = (i - 1) // 2
        if heap[i] > heap[parent]:
            heap[i], heap[parent] = heap[parent], heap[i]
            i = parent
        else:
            break

def delete(heap):
    if len(heap) == 0:
        print("Heap is empty")
        return
    print("Deleted element:", heap[0])
    heap[0] = heap[-1]
    heap.pop()

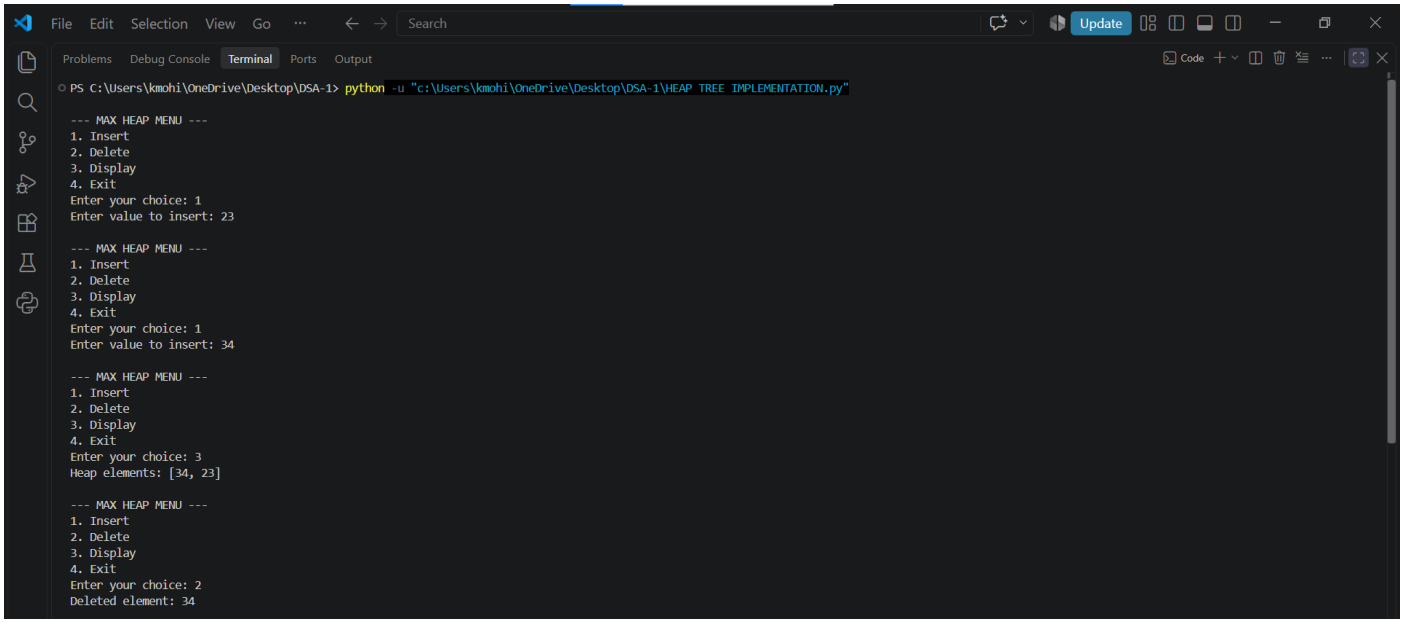
    n = len(heap)
    i = 0
    # Move the root downward
    while True:
        largest = i
        left = 2 * i + 1
        right = 2 * i + 2
        if left < n and heap[left] > heap[largest]:
            largest = left
        if right < n and heap[right] > heap[largest]:
            largest = right
        if largest != i:
            heap[i], heap[largest] = heap[largest], heap[i]
            i = largest
        else:
            break

def display(heap):
    if len(heap) == 0:
        print("Heap is empty")
    else:
        print("Heap elements:", heap)

#Main Program
heap = []
while True:
    print("\n--- MAX HEAP MENU ---")
```

```
    print("1. Insert")
    print("2. Delete")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        value = int(input("Enter value to insert: "))
        insert(heap, value)
    elif choice == 2:
        delete(heap)
    elif choice == 3:
        display(heap)
    elif choice == 4:
        print("Exiting program")
        break
    else:
        print("Invalid choice")
```

Output:



WEEK 12

Aim:

Write a program to implement the following three collision resolution techniques in a Hash Table:

4. Separate Chaining
5. Linear Probing
6. Quadratic Probing

Program (1.seperate chaining):

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

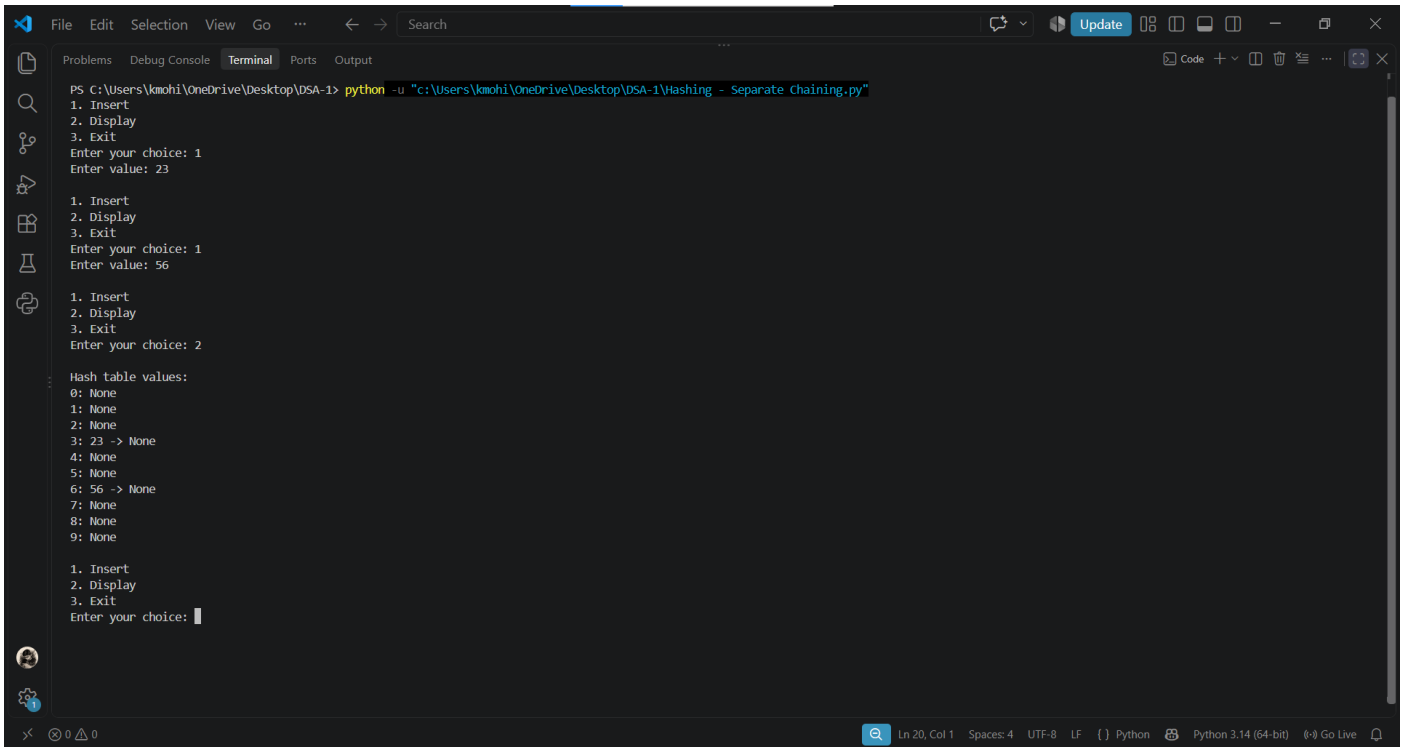
SIZE = 10
ht = [None] * SIZE


def insertValue(value):
    index = value % SIZE
    newnode = Node(value)
    if ht[index] is None:
        ht[index] = newnode
    else:
        temp = ht[index]
        while temp.next is not None:
            temp = temp.next
        temp.next = newnode


def displayValue():
    for i in range(SIZE):
        print(i, end=": ")
        temp = ht[i]
        if temp is None:
            print("None")
        else:
            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")

repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        print("\nHash table values:")
        displayValue()
    elif ch == 3:
        repeat = False
    else:
        print("Invalid choice")
```

Output:



Program(2.Linear probing):

```
SIZE = 10
a = [-1] * SIZE
def insertValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == -1:
            a[index] = x
            print("Element inserted")
            return
    print("Hash table is full")


def deleteValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            a[index] = -1
            print("Element deleted")
            return
    print("Element not found")

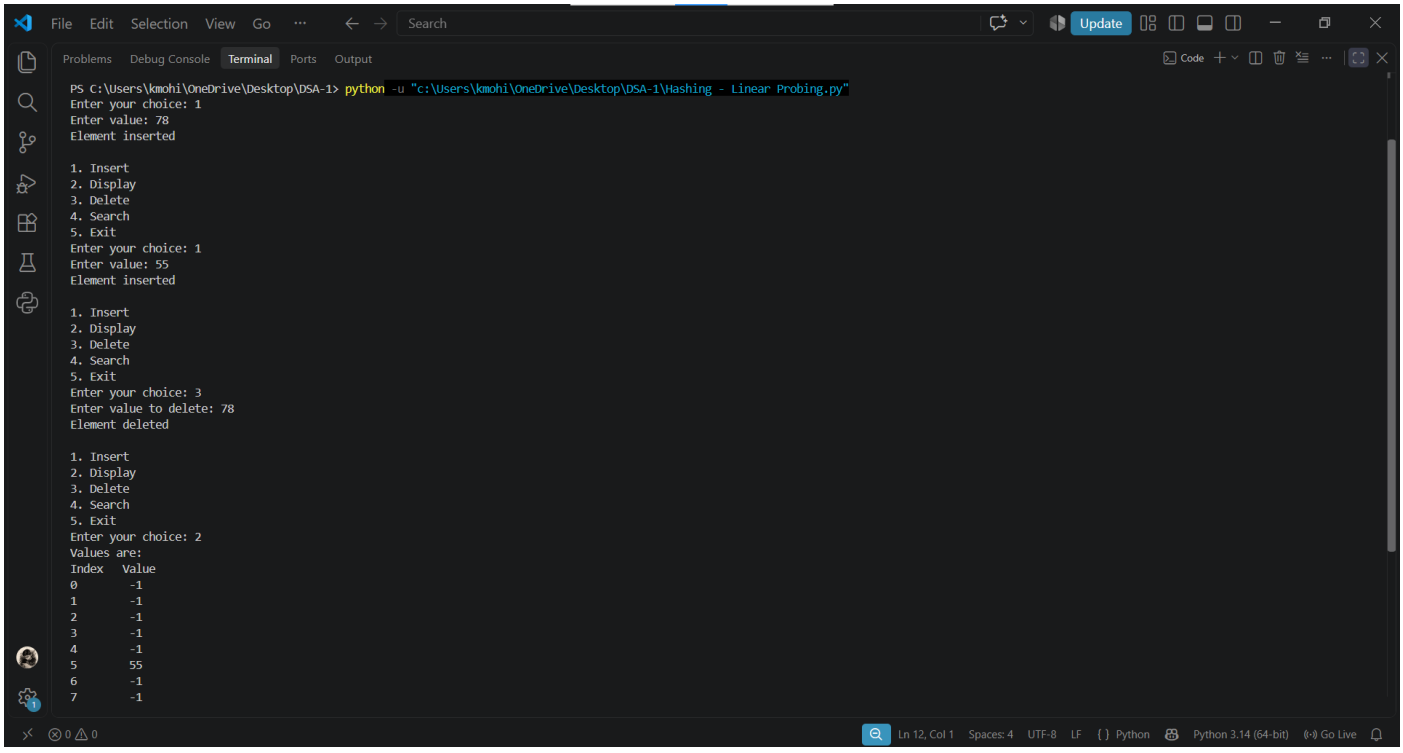
def search(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            print("Element found at index", index)
            return
        if a[index] == -1:
            break
    print("Element not found")


def displayValue():
```

```
    print("Index\tValue")
    for i in range(SIZE):
        print(i, "\t", a[i])

# Menu-driven program
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Delete")
    print("4. Search")
    print("5. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        print("Values are:")
        displayValue()
    elif ch == 3:
        value = int(input("Enter value to delete: "))
        deleteValue(value)
    elif ch == 4:
        value = int(input("Enter value to search: "))
        search(value)
    elif ch == 5:
        repeat = False
    else:
        print("Invalid choice")
```

Output:



Program(3.Quadratic probing):

```
SIZE = 10
a = [-1] * SIZE

def insertValue(value):
    for i in range(SIZE):
        index = ((value % SIZE) + i * i) % SIZE
    if a[index] == -1:
            a[index] = value
            print("Element inserted at index", index)
            return
    print("No empty slot found in the probe sequence")


def displayValue():
    print("Values are:")
    for i in range(SIZE):
        print(a[i])

# Menu-driven program
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        displayValue()
    elif ch == 3:
        repeat = False
    else:
        print("Invalid choice")
```

Output:

